



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ

ΕΡΓΑΣΙΑ 2 ΚΡΥΠΤΟΓΡΑΦΙΑ

ΣΤΟΙΧΕΙΑ ΦΟΙΤΗΤΗ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ : ΑΘΑΝΑΣΙΟΥ ΒΑΣΙΛΕΙΟΣ ΕΥΑΓΓΕΛΟΣ
ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ : 19390005
ΕΞΑΜΗΝΟ ΦΟΙΤΗΤΗ : 8^ο
ΚΑΤΑΣΤΑΣΗ ΦΟΙΤΗΤΗ : ΠΡΟΠΤΥΧΙΑΚΟ
ΠΡΟΓΡΑΜΜΑ ΣΠΟΥΔΩΝ : ΠΑΔΑ
ΤΜΗΜΑ ΕΡΓΑΣΤΗΡΙΟΥ : ΑΣΦ 05 ΤΕΤΑΡΤΗ 12:00 – 14:00
ΥΠΕΥΘΥΝΟΣ ΕΡΓΑΣΤΗΡΙΟΥ : ΓΕΩΡΓΟΥΛΑΣ ΑΓΓΕΛΟΣ
ΗΜΕΡΟΜΗΝΙΑ ΠΑΡΑΔΟΣΗΣ : 4/5/2023

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΦΩΤΟΓΡΑΦΙΑ ΦΟΙΤΗΤΗ :



ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΠΕΡΙΕΧΟΜΕΝΑ

ΔΡΑΣΤΗΡΙΟΤΗΤΕΣ	(ΣΕΛΙΔΕΣ 4 – 40)
<u>1. Δημιουργία ιδιωτικού κλειδιού</u>	<u>(ΣΕΛΙΔΕΣ 4 – 7)</u>
1.1 Ενέργειες	(ΣΕΛΙΔΕΣ 4 – 6)
1.2 Αποτελέσματα	(ΣΕΛΙΔΑ 7)
1.3 Παρατηρήσεις	(ΣΕΛΙΔΑ 7)
<u>2. Κρυπτογράφηση μηνύματος</u>	<u>(ΣΕΛΙΔΕΣ 7 – 12)</u>
2.1 Ενέργειες	(ΣΕΛΙΔΕΣ 7 – 10)
2.2 Αποτελέσματα	(ΣΕΛΙΔΕΣ 11 – 12)
2.3 Παρατηρήσεις	(ΣΕΛΙΔΑ 12)
<u>3. Αποκρυπτογράφηση μηνύματος</u>	<u>(ΣΕΛΙΔΕΣ 12 – 16)</u>
3.1 Ενέργειες	(ΣΕΛΙΔΕΣ 12 – 15)
3.2 Αποτελέσματα	(ΣΕΛΙΔΕΣ 15 – 16)
3.3 Παρατηρήσεις	(ΣΕΛΙΔΑ 16)
<u>4. Υπογραφή μηνύματος</u>	<u>(ΣΕΛΙΔΕΣ 16 – 22)</u>
4.1 Ενέργειες	(ΣΕΛΙΔΕΣ 16 – 19)
4.2 Αποτελέσματα	(ΣΕΛΙΔΕΣ 20 – 22)
4.3 Παρατηρήσεις	(ΣΕΛΙΔΑ 22)
<u>5. Επαλήθευση Υπογραφής</u>	<u>(ΣΕΛΙΔΕΣ 23 – 29)</u>
5.1 Ενέργειες	(ΣΕΛΙΔΕΣ 23 – 26)
5.2 Αποτελέσματα	(ΣΕΛΙΔΕΣ 26 – 29)
5.3 Παρατηρήσεις	(ΣΕΛΙΔΑ 29)
<u>6. Μη αυτόματη επαλήθευση πιστοποιητικού X.509</u>	<u>(ΣΕΛΙΔΕΣ 29 – 40)</u>
6.1 Ενέργειες	(ΣΕΛΙΔΕΣ 29 – 34)
6.2 Αποτελέσματα	(ΣΕΛΙΔΕΣ 34 – 40)
6.3 Παρατηρήσεις	(ΣΕΛΙΔΑ 40)

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΔΡΑΣΤΗΡΙΟΤΗΤΕΣ

1. Δημιουργία ιδιωτικού κλειδιού

1.1 Ενέργειες

Σκοπός της δραστηριότητας ήταν η δημιουργία του ιδιωτικού κλειδιού όπου αυτό επιτυγχάνεται χάρις στην βιβλιοθήκη συναρτήσεων BN (BIGNUM), όπου χειρίζεται αριθμούς που ξεφεύγουν από τα όρια της αρχιτεκτονικής υπολογιστών και εκτελεί μαθηματικές πράξεις με αυτούς (προσθέσεις, αφαιρέσεις, πολλαπλασιασμούς κλπ.). Το `private_key.c` (Εικόνα 1.1.1) υπολογίζει και εκτυπώνει το ιδιωτικό κλειδί, υλοποιώντας κάποιες πράξεις. Πιο αναλυτικά:

στις γραμμές 26 και 30 αρχικοποιεί δύο μεγάλους αριθμούς στις δομές BN, `p` και `q`

```
BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");  
BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
```

στη γραμμή 38 αρχικοποιεί τον ακέραιο αριθμό 1 στην δομή BN, `one`

```
BN_dec2bn(&one, "1");
```

στη γραμμή 34 αρχικοποιεί το δημόσιο κλειδί στην δομή BN, `e`

```
BN_hex2bn(&e, "0D88C3");
```

στις γραμμές 43 και 48 υπολογίζει τις διαφορές $p - 1$ και $q - 1$ και εκχωρεί τα αποτελέσματα στις δομές BN, `p1` και `q1` αντίστοιχα

```
BN_sub(p1, p, one);  
BN_sub(q1, q, one);
```

στη γραμμή 53 υπολογίζει το γινόμενο $(p - 1) * (q - 1)$ και εκχωρεί το αποτέλεσμα στην δομή BN, `phi`, δηλαδή, υπολογίζει τον αριθμό $\Phi(N)$

```
BN_mul(phi, p1, q1, ctx);
```

στη γραμμή 58 υπολογίζει το γινόμενο $p * q$ και εκχωρεί το αποτέλεσμα στη δομή BN, `N`, δηλαδή, υπολογίζει το modulo N

```
BN_mul(N, p, q, ctx);
```

στη γραμμή 63 υπολογίζει τον αντίστροφο της αριθμητικής υπολοίπου $e * d \bmod \Phi(N) = 1$ στη δομή BN, `d`, δηλαδή, υπολογίζει το ιδιωτικό κλειδί

```
BN_mod_inverse(d, e, phi, ctx);
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
private_key.c
1  #include <stdio.h>
2  #include <openssl/bn.h>
3
4  #define NBITS 256
5
6  void printBN(char *msg, BIGNUM *a);
7
8  int main(int argc, char *argv[])
9  {
10 /*
11  * Δημιουργία των δομών τύπου BN
12  */
13  BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14  BIGNUM *p = BN_new();       // Μεγάλος αριθμός p
15  BIGNUM *q = BN_new();       // Μεγάλος αριθμός q
16  BIGNUM *p1 = BN_new();      // Αφαίρεση p - 1
17  BIGNUM *q1 = BN_new();      // Αφαίρεση q - 1
18  BIGNUM *one = BN_new();     // Ακέραιος 1
19  BIGNUM *phi = BN_new();     // Αριθμός  $\phi(N) = (p - 1) * (q - 1)$ 
20  BIGNUM *N = BN_new();      // Modulo N
21  BIGNUM *e = BN_new();      // Δημόσιο κλειδί (e, N)
22  BIGNUM *d = BN_new();      // Ιδιωτικό κλειδί (d, N)
23
24  /* Αρχικοποίηση του πρώτου αριθμού p */
25  BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
26
27  /* Αρχικοποίηση του πρώτου αριθμού q */
28  BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
29
30  /* Αρχικοποίηση του δημόσιου κλειδιού e ώστε να είναι πρώτος με το  $\phi(N)$  */
31  BN_hex2bn(&e, "0D88C3");
32
33  /* Μετατροπή του ακέραιου αριθμού 1 σε δομή τύπου BN */
34  BN_dec2bn(&one, "1");
35
36  /* Υπολογισμός του πρώτου όρου του  $\phi(N)$  */
37  BN_sub(p1, p, one);
38
39  /* Υπολογισμός της αφαίρεσης q - 1 */
40  BN_sub(q1, q, one);
41
42  /* Υπολογισμός του αριθμού  $\phi(N)$  */
43  BN_mul(phi, p1, q1, ctx);
44
45  /* Υπολογισμός του modulo N */
46  BN_mul(N, p, q, ctx);
47
48  /* Υπολογισμός του γινομένου  $N = p * q$  */
49
50  /* Υπολογισμός του ιδιωτικού κλειδιού (d, N) */
51  BN_mod_inverse(d, e, phi, ctx);
52
53  /* Υπολογισμός του αντίστροφου σε αριθμητική υπολοίπου,  $e * d \bmod \phi(N) = 1$  */
54
55  /* Εκτύπωση του modulo  $N = p * q$  */
56  printBN("N =\t", N);
57
58  /* Εκτύπωση του αριθμού  $\phi(N) = (p - 1) * (q - 1)$  */
59  printBN("Phi =\t", phi);
60
61  /* Εκτύπωση του ιδιωτικού κλειδιού d */
62  printBN("d=\t", d);
63
64  }
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
76  /*
77  * Αποδέσμευση των δομών τύπου BN από την μνήμη
78  */
79  BN_CTX_free(ctx);
80  BN_free(p);
81  BN_free(q);
82  BN_free(p1);
83  BN_free(q1);
84  BN_free(one);
85  BN_free(phi);
86  BN_free(N);
87  BN_free(e);
88  BN_free(d);
89
90  return 0;
91  }
92
93  /*
94  * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
95  */
96  void printBN(char *msg, BIGNUM *a)
97  {
98      char *number_str = BN_bn2hex(a);
99      printf("%s %s\n", msg, number_str);
100      OPENSSL_free(number_str);
101  }
```

Εικόνα 1.1.1. Το πρόγραμμα private_key.c

Στη συνέχεια, μεταγλωττίσαμε τον κώδικα με την παράμετρο

```
-lcrypto
```

όπου συνδέει την βιβλιοθήκη συναρτήσεων χειρισμού μεγάλων αριθμών, BN της OpenSSL.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc private_key.c -o private_key -lcrypto
```

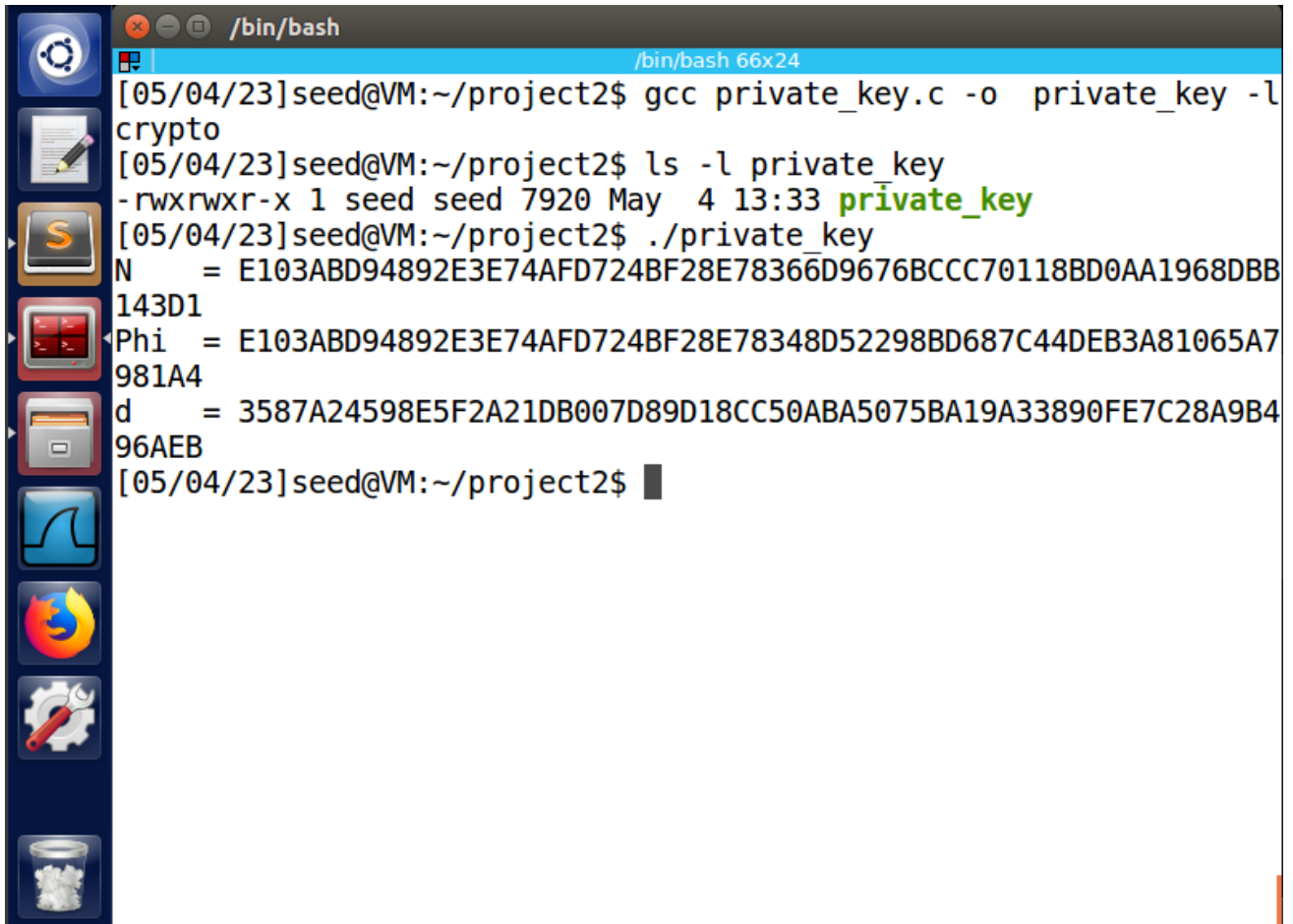
και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο private_key.

Τέλος, τρέξαμε το πρόγραμμα private_key με την εντολή :

```
./private_key
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

1.2 Αποτελέσματα



```
/bin/bash
[05/04/23]seed@VM:~/project2$ gcc private_key.c -o private_key -l
crypto
[05/04/23]seed@VM:~/project2$ ls -l private_key
-rwxrwxr-x 1 seed seed 7920 May  4 13:33 private_key
[05/04/23]seed@VM:~/project2$ ./private_key
N      = E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB
143D1
Phi    = E103ABD94892E3E74AFD724BF28E78348D52298BD687C44DEB3A81065A7
981A4
d      = 3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B4
96AEB
[05/04/23]seed@VM:~/project2$
```

Εικόνα 1.2.1 Μεταγλώττιση και εκτέλεση του προγράμματος private_key.c

1.3 Παρατηρήσεις

Το modulo N υπολογίστηκε από τον μαθηματικό τύπο $N = p * q$, το $\Phi(N)$ από τον τύπο $\Phi(N) = (p - 1) * (q - 1)$ και το ιδιωτικό κλειδί d από τον τύπο $d = e * d \bmod \Phi(N) = 1$. Το δημόσιο κλειδί e είναι πρώτος αριθμός με το $\Phi(N)$.

2. Κρυπτογράφηση μηνύματος

2.1 Ενέργειες

Στην δραστηριότητα αυτή κρυπτογραφήσαμε ένα μήνυμα χάρις στο πρόγραμμα crypto_msg.c. Το μήνυμα είναι το όνομα του σπουδαστή «Vasilis Athanasiou» και αρχικά το κωδικοποιήσαμε σε δεκαεξαδική μορφή με το python script

```
python -c 'print("Vasilis Athanasiou".encode("hex"))'
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Το `crypto_msg.c` (Εικόνα 2.1.1) κρυπτογραφεί το μήνυμα αυτό υλοποιώντας μαθηματικές πράξεις με μεγάλους αριθμούς χάρις στην βιβλιοθήκη συναρτήσεων BN που χειρίζεται αριθμούς που ξεφεύγουν από τα όρια της αρχιτεκτονικής υπολογιστών. Πιο αναλυτικά:

στη γραμμή 24 αρχικοποιεί το δημόσιο κλειδί στην δομή BN, e

```
BN_hex2bn(&e, "0D88C3");
```

στη γραμμή 29 αρχικοποιεί το modulo στην δομή BN, N

```
BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
```

στη γραμμή 34 αρχικοποιεί το ιδιωτικό κλειδί στην δομή BN, d

```
BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
```

στη γραμμή 39 αρχικοποιεί το κωδικοποιημένο μήνυμα «Vasilis Athanasiou» στην δομή BN, M

```
BN_hex2bn(&M, "566173696C697320417468616E6173696F75");
```

στη γραμμή 44 υπολογίζει την μαθηματική πράξη $M^e \bmod N$ και εκχωρεί το αποτέλεσμα στη δομή BN, C , δηλαδή, κρυπτογραφεί το μήνυμα «Vasilis Athanasiou» (δομή M)

```
BN_mod_exp(C, M, e, N, ctx);
```

στη γραμμή 53 υπολογίζει την μαθηματική πράξη $C^d \bmod N$ και εκχωρεί το αποτέλεσμα στη δομή BN, D , δηλαδή, αποκρυπτογραφεί το μήνυμα «Vasilis Athanasiou» (δομή M)

```
BN_mod_exp(D, C, d, N, ctx);
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
crypto_msg.c x
1 |#include <stdio.h>
2 |#include <openssl/bn.h>
3
4 |#define NBITS 256
5
6 |void printBN(char *msg, BIGNUM *a);
7
8 |int main(int argc, char *argv[])
9 |{
10 |/*
11 | * Δημιουργία των δομών τύπου BN
12 | */
13 |    BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14 |    BIGNUM *M = BN_new(); // Μήνυμα με το ονοματεπώνυμο του σπουδαστή
15 |    BIGNUM *C = BN_new(); // Κρυπτογραφημένο μήνυμα
16 |    BIGNUM *D = BN_new(); // Αποκρυπτογραφημένο μήνυμα
17 |    BIGNUM *e = BN_new(); // Δημόσιο κλειδί (e, N)
18 |    BIGNUM *N = BN_new(); // Μόδulo N
19 |    BIGNUM *d = BN_new(); // Ιδιωτικό κλειδί (d, N)
20 |/*
21 | * Αρχικοποίηση του δημόσιου κλειδιού (e, N) που χρησιμοποιήθηκε στην
22 | * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
23 | */
24 |    BN_hex2bn(&e, "0D88C3");
25 |/*
26 | * Αρχικοποίηση του modulo N που υπολογίστηκε στην
27 | * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
28 | */
29 |    BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
30 |/*
31 | * Αρχικοποίηση του δημόσιου κλειδιού (d, N) που υπολογίστηκε στην
32 | * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
33 | */
34 |    BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
35 |/*
36 | * Αρχικοποίηση του μηνύματος με το ονοματεπώνυμο του σπουδαστή
37 | * "Vasilis Athanasiou"
38 | */
39 |    BN_hex2bn(&M, "566173696C6973204174686616E6173696F75");
40 |/*
41 | * Υπολογισμός του κρυπτογραφημένου μηνύματος με τον τύπο
42 | *  $C = M^e \bmod N$ 
43 | */
44 |    BN_mod_exp(C, M, e, N, ctx);
45 |/*
46 | * Εκτύπωση του κρυπτογραφημένου μηνύματος
47 | */
48 |    printBN("Encrypted Message =", C);
49 |/*
50 | * Υπολογισμός του αποκρυπτογραφημένου μηνύματος με τον τύπο
51 | *  $D = C^d \bmod N$ 
52 | */
53 |    BN_mod_exp(D, C, d, N, ctx);
54 |/*
55 | * Εκτύπωση του αποκρυπτογραφημένου μηνύματος
56 | */
57 |    printBN("Decrypted Message = ", D);
58 |/*
59 | * Αποδέσμευση των δομών τύπου BN από την μνήμη
60 | */
61 |    BN_CTX_free(ctx);
62 |    BN_free(M);
63 |    BN_free(C);
64 |    BN_free(D);
65 |    BN_free(e);
66 |    BN_free(N);
67 |    BN_free(d);
68
69 |    return 0;
70 |}
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
71
72 /*
73  * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
74  */
75 void printBN(char *msg, BIGNUM *a)
76 {
77     char *number_str = BN_bn2hex(a);
78     printf("%s %s\n", msg, number_str);
79     OPENSSL_free(number_str);
80 }
```

Εικόνα 2.1.1 Το πρόγραμμα crypto_msg.c

Στη συνέχεια, μεταγλωττίσαμε τον κώδικα με την παράμετρο

```
-lcrypto
```

όπου συνδέει την βιβλιοθήκη συναρτήσεων χειρισμού μεγάλων αριθμών, BN της OpenSSL.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc crypto_msg.c -o crypto_msg -lcrypto
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο crypto_msg.

Στη συνέχεια, τρέξαμε το πρόγραμμα crypto_msg με την εντολή :

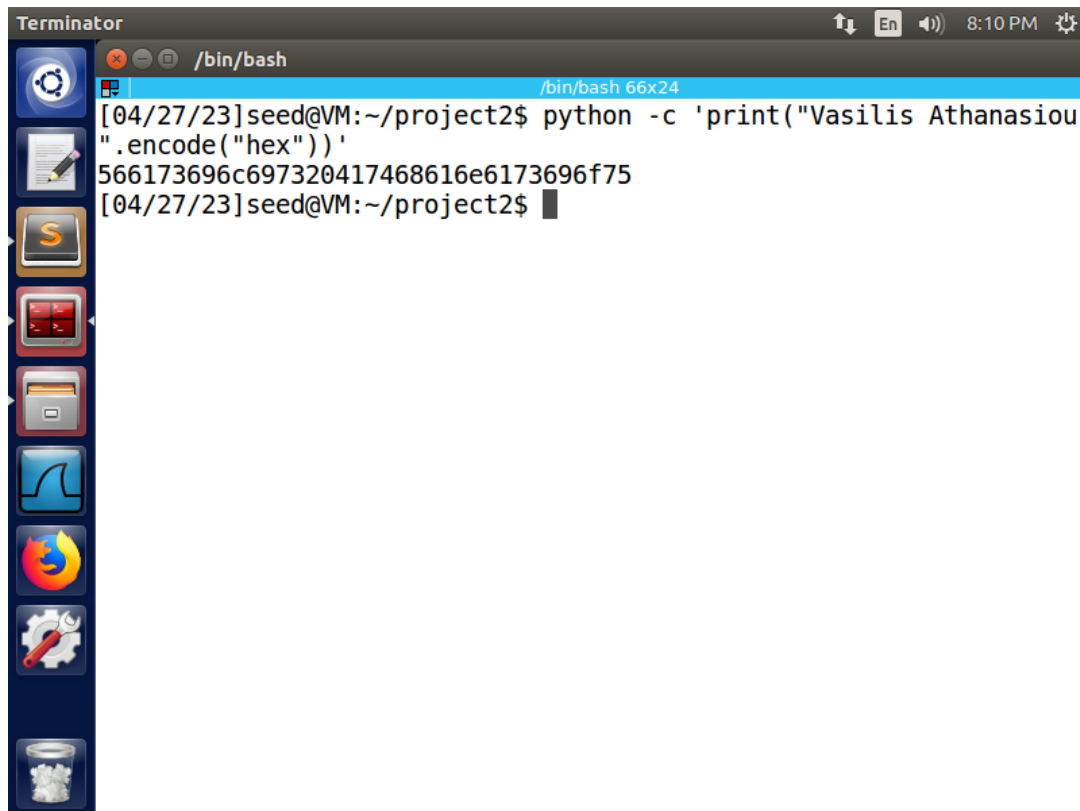
```
./crypto_msg
```

Τέλος, για να επαληθεύσουμε ότι το πρόγραμμα crypto_msg.c κρυπτογραφεί το μήνυμα «Vasilis Athanasiou», εκτέλεσαμε ένα script της python, όπου αποκωδικοποιεί την συμβολοσειρά από δεκαεξαδική μορφή σε αναγνώσιμη μορφή. Η εντολή είναι :

```
python -c 'print("566173696C697320417468616E6173696F75".decode("hex"))'
```

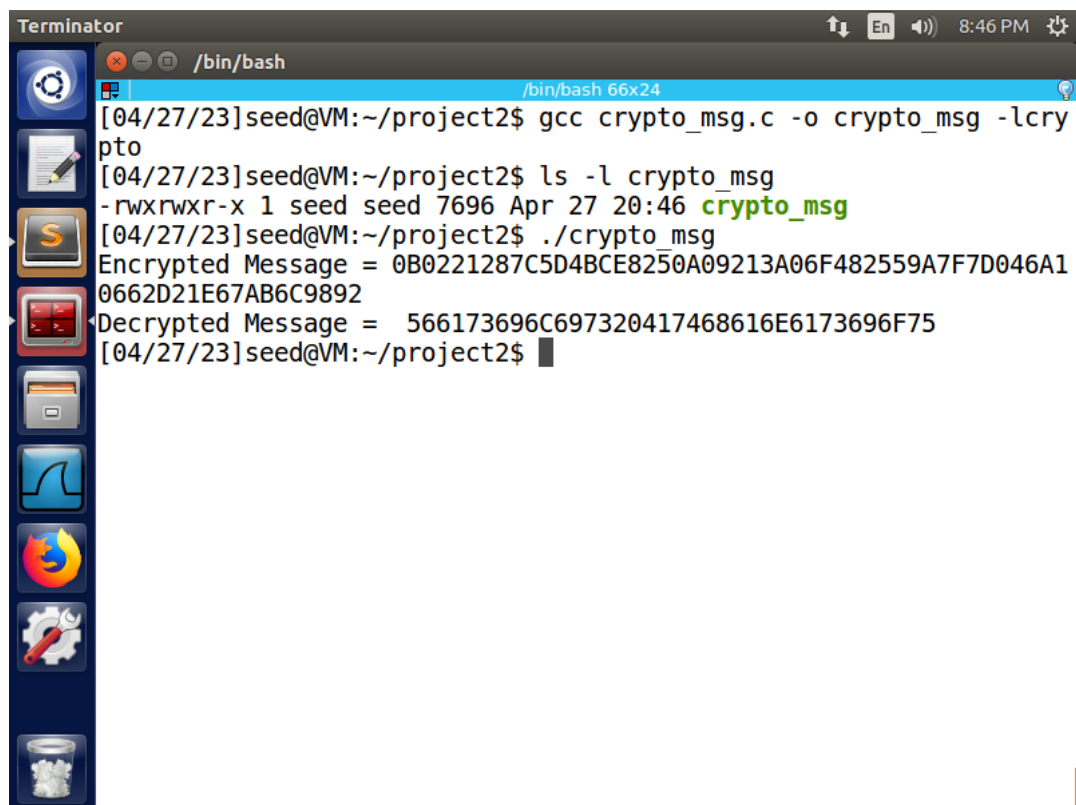
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

2.2 Αποτελέσματα



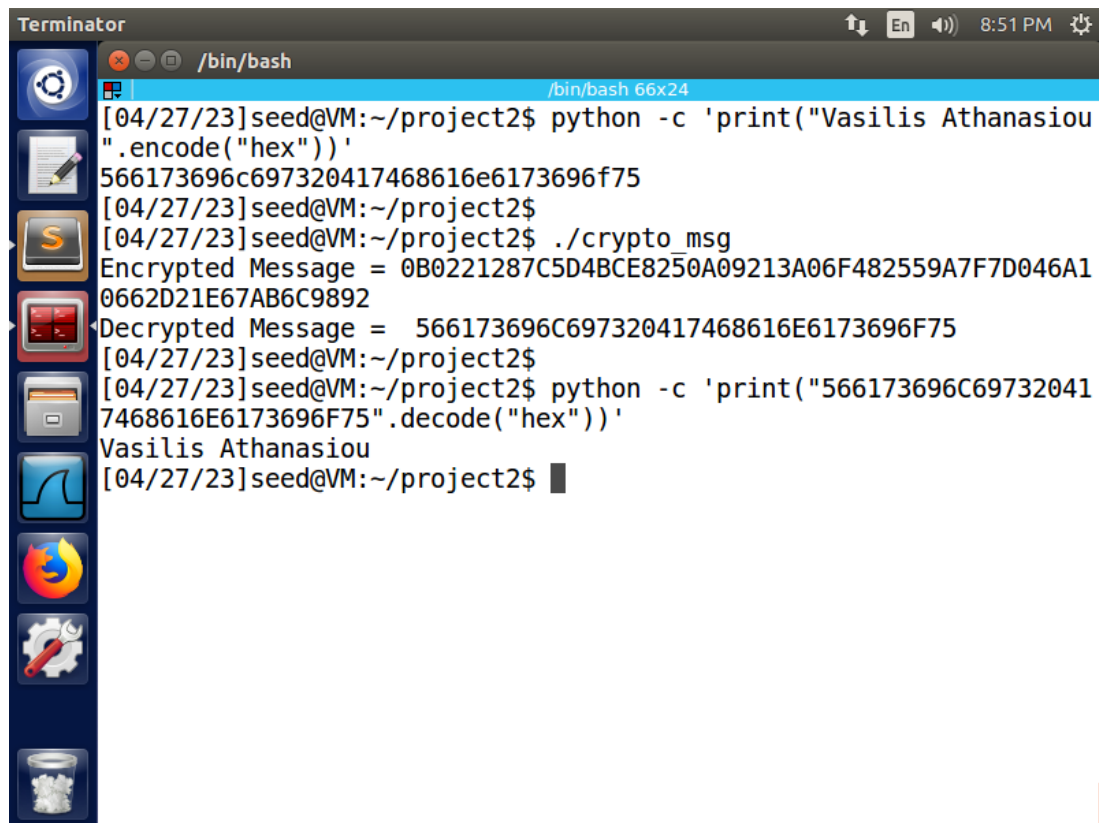
```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Vasilis Athanasiou".encode("hex"))'
566173696c697320417468616e6173696f75
[04/27/23]seed@VM:~/project2$
```

Εικόνα 2.2.1. Κωδικοποίηση του μηνύματος «Vasilis Athanasiou» σε δεκαεξαδική μορφή



```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ gcc crypto_msg.c -o crypto_msg -lcrypto
[04/27/23]seed@VM:~/project2$ ls -l crypto_msg
-rwxrwxr-x 1 seed seed 7696 Apr 27 20:46 crypto_msg
[04/27/23]seed@VM:~/project2$ ./crypto_msg
Encrypted Message = 0B0221287C5D4BCE8250A09213A06F482559A7F7D046A10662D21E67AB6C9892
Decrypted Message = 566173696C697320417468616E6173696F75
[04/27/23]seed@VM:~/project2$
```

Εικόνα 2.2.2 Μεταγλώττιση και εκτέλεση του προγράμματος crypto_msg.c



```
Terminator                                     ↑ ↓  En  8:51 PM  ⚙
/bin/bash
[04/27/23]seed@VM:~/project2$ python -c 'print("Vasilis Athanasiou".encode("hex"))'
566173696c697320417468616e6173696f75
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ ./crypto_msg
Encrypted Message = 0B0221287C5D4BCE8250A09213A06F482559A7F7D046A10662D21E67AB6C9892
Decrypted Message = 566173696C697320417468616E6173696F75
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ python -c 'print("566173696C697320417468616E6173696F75".decode("hex"))'
Vasilis Athanasiou
[04/27/23]seed@VM:~/project2$
```

Εικόνα 2.2.3. Επαλήθευση της επιτυχημένης κρυπτογράφησης του μηνύματος «Vasilis Athanasiou»

2.3 Παρατηρήσεις

Η κρυπτογράφηση του μηνύματος υπολογίστηκε από τον μαθηματικό τύπο $C = M^e \bmod N$. Η αποκρυπτογράφηση του μηνύματος υπολογίστηκε από τον μαθηματικό τύπο $D = C^d \bmod N$. Το python script αποκωδικοποίησης (decode) της συμβολοσειράς από δεκαεξαδική σε αναγνώσιμη μορφή, παίρνοντας σαν όρισμα την δομή D που υπολόγισε το πρόγραμμα crypto_msg.c, τύπωσε στο output το αρχικό μήνυμα (Vasilis Athanasiou) που βάλαμε ως είσοδο στην δομή M.

3. Αποκρυπτογράφηση μηνύματος

3.1 Ενέργειες

Στην δραστηριότητα αυτή αποκρυπτογραφήσαμε ένα μήνυμα χάρις στο πρόγραμμα decrypto_msg.c. Το μήνυμα αυτό είναι κρυπτογραφημένο σε δεκαεξαδική μορφή

```
C = 7074A77B724869F59EBA8B790CEAA6A33B2E268A00BAF124BA7C3A6BDE690F36
```

και σκοπός της δραστηριότητας ήταν να το αποκρυπτογραφήσουμε και στην συνέχεια να το μετατρέψουμε σε αναγνώσιμη μορφή με το python script.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Το `decrypt_msg.c` (Εικόνα 3.1.1) αποκρυπτογραφεί το μήνυμα αυτό υλοποιώντας μαθηματικές πράξεις με μεγάλους αριθμούς χάρις στην βιβλιοθήκη συναρτήσεων BN που χειρίζεται αριθμούς που ξεφεύγουν από τα όρια της αρχιτεκτονικής υπολογιστών. Πιο αναλυτικά:

στη γραμμή 23 αρχικοποιεί το δημόσιο κλειδί στην δομή BN, `e`

```
BN_hex2bn (&e, "0D88C3");
```

στη γραμμή 28 αρχικοποιεί το modulo στην δομή BN, `N`

```
BN_hex2bn (&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
```

στη γραμμή 33 αρχικοποιεί το ιδιωτικό κλειδί στην δομή BN, `d`

```
BN_hex2bn (&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
```

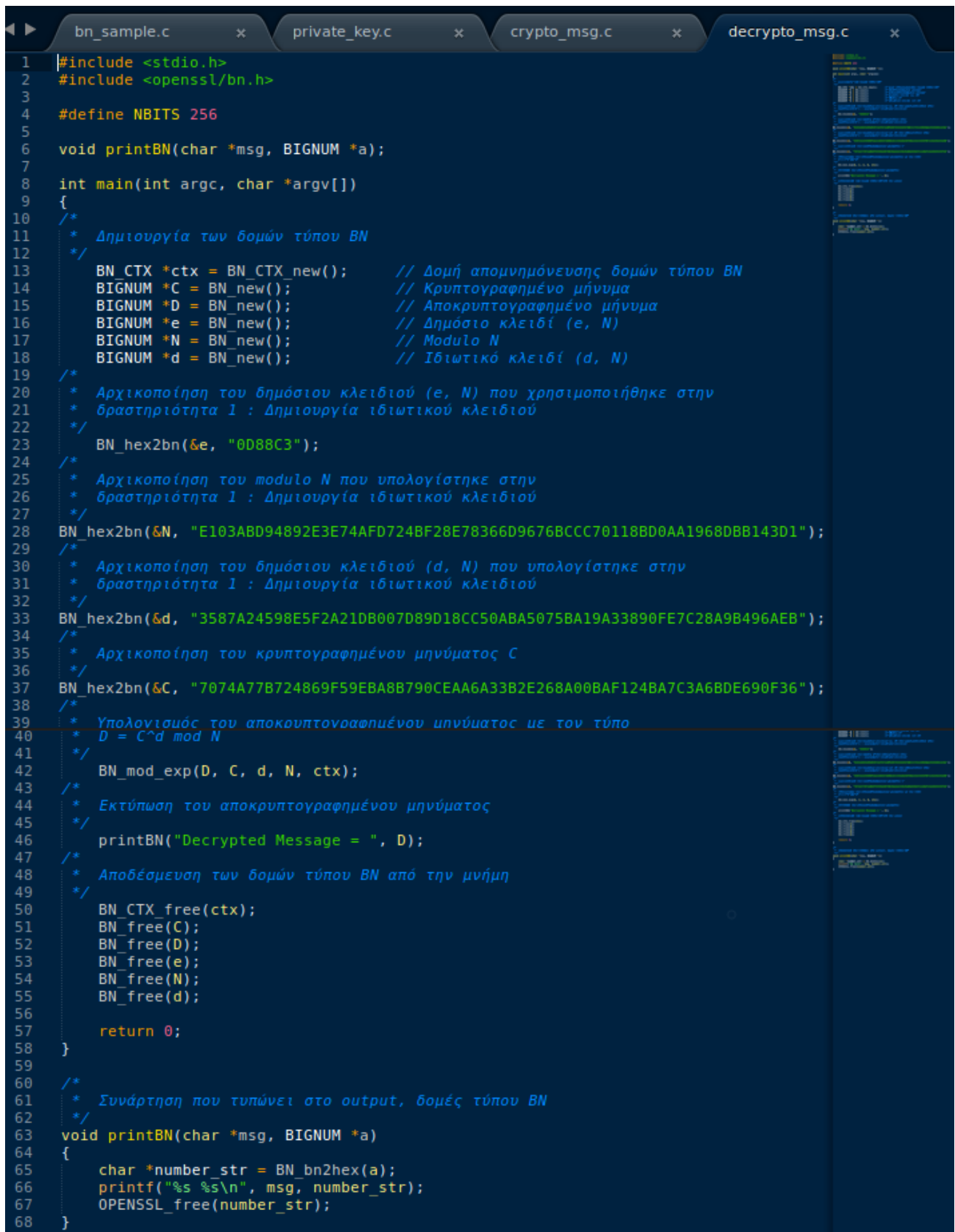
στη γραμμή 37 αρχικοποιεί το κρυπτογραφημένο μήνυμα, κωδικοποιημένο σε δεκαεξαδική μορφή στην δομή BN, `C`

```
BN_hex2bn (&C, "7074A77B724869F59EBA8B790CEAA6A33B2E268A00BAF124BA7C3A6BDE690F36");
```

στη γραμμή 42 υπολογίζει την μαθηματική πράξη $C^d \bmod N$ και εκχωρεί το αποτέλεσμα στη δομή BN, `D`, δηλαδή, αποκρυπτογραφεί το μήνυμα `C` που είναι κωδικοποιημένο σε δεκαεξαδική μορφή

```
BN_mod_exp (D, C, d, N, ctx);
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
1  #include <stdio.h>
2  #include <openssl/bn.h>
3
4  #define NBITS 256
5
6  void printBN(char *msg, BIGNUM *a);
7
8  int main(int argc, char *argv[])
9  {
10     /*
11     * Δημιουργία των δομών τύπου BN
12     */
13     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14     BIGNUM *C = BN_new();        // Κρυπτογραφημένο μήνυμα
15     BIGNUM *D = BN_new();        // Αποκρυπτογραφημένο μήνυμα
16     BIGNUM *e = BN_new();        // Δημόσιο κλειδί (e, N)
17     BIGNUM *N = BN_new();        // Modulo N
18     BIGNUM *d = BN_new();        // Ιδιωτικό κλειδί (d, N)
19
20     /*
21     * Αρχικοποίηση του δημόσιου κλειδιού (e, N) που χρησιμοποιήθηκε στην
22     * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
23     */
24     BN_hex2bn(&e, "0D88C3");
25
26     /*
27     * Αρχικοποίηση του modulo N που υπολογίστηκε στην
28     * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
29     */
30     BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
31
32     /*
33     * Αρχικοποίηση του δημόσιου κλειδιού (d, N) που υπολογίστηκε στην
34     * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
35     */
36     BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
37
38     /*
39     * Αρχικοποίηση του κρυπτογραφημένου μηνύματος C
40     */
41     BN_hex2bn(&C, "7074A77B724869F59EBA8B790CEAA6A33B2E268A00BAF124BA7C3A6BDE690F36");
42
43     /*
44     * Υπολογισμός του αποκρυπτογραφημένου μηνύματος με τον τύπο
45     *  $D = C^d \text{ mod } N$ 
46     */
47     BN_mod_exp(D, C, d, N, ctx);
48
49     /*
50     * Εκτύπωση του αποκρυπτογραφημένου μηνύματος
51     */
52     printBN("Decrypted Message = ", D);
53
54     /*
55     * Αποδέσμευση των δομών τύπου BN από την μνήμη
56     */
57     BN_CTX_free(ctx);
58     BN_free(C);
59     BN_free(D);
60     BN_free(e);
61     BN_free(N);
62     BN_free(d);
63
64     return 0;
65 }
66
67 /*
68 * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
69 */
70 void printBN(char *msg, BIGNUM *a)
71 {
72     char *number_str = BN_bn2hex(a);
73     printf("%s %s\n", msg, number_str);
74     OPENSSL_free(number_str);
75 }
```

Εικόνα 3.1.1 Το πρόγραμμα decrypto_msg.c

Στη συνέχεια, μεταγλωττίσαμε τον κώδικα με την παράμετρο

```
-lcrypto
```

όπου συνδέει την βιβλιοθήκη συναρτήσεων χειρισμού μεγάλων αριθμών, BN της OpenSSL.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc decrypto_msg.c -o decrypto_msg -lcrypto
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο `decrypto_msg`.

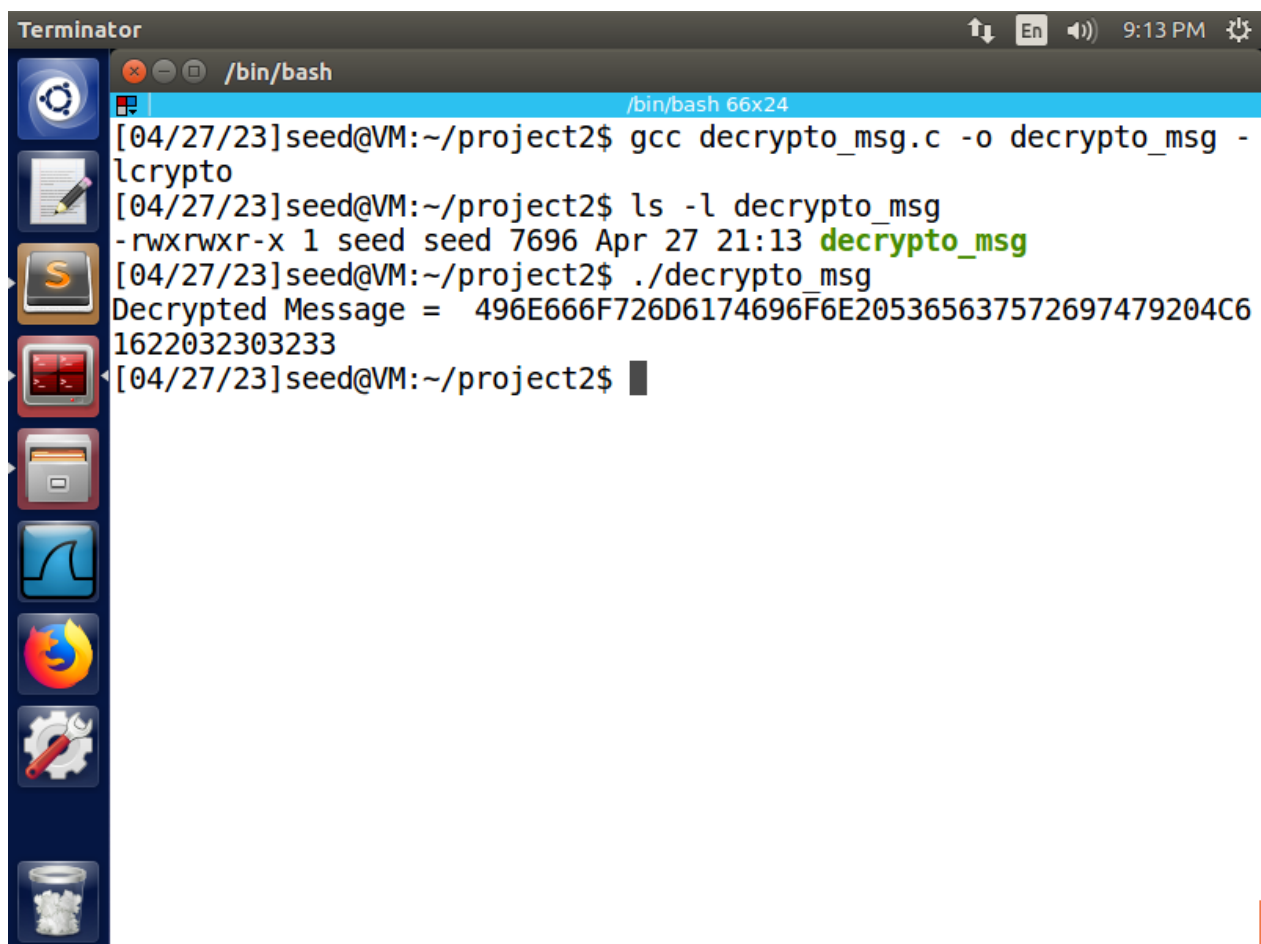
Στη συνέχεια, τρέξαμε το πρόγραμμα `decrypto_msg` με την εντολή :

```
./decrypto_msg
```

Τέλος, για να επαληθεύσουμε ότι το πρόγραμμα `decrypto_msg.c` αποκρυπτογραφεί το μήνυμα που είναι κρυπτογραφημένο και κωδικοποιημένο σε δεκαεξαδική μορφή, εκτέλεσαμε ένα script της python, όπου αποκωδικοποιεί την συμβολοσειρά από δεκαεξαδική μορφή σε αναγνώσιμη μορφή. Η εντολή είναι :

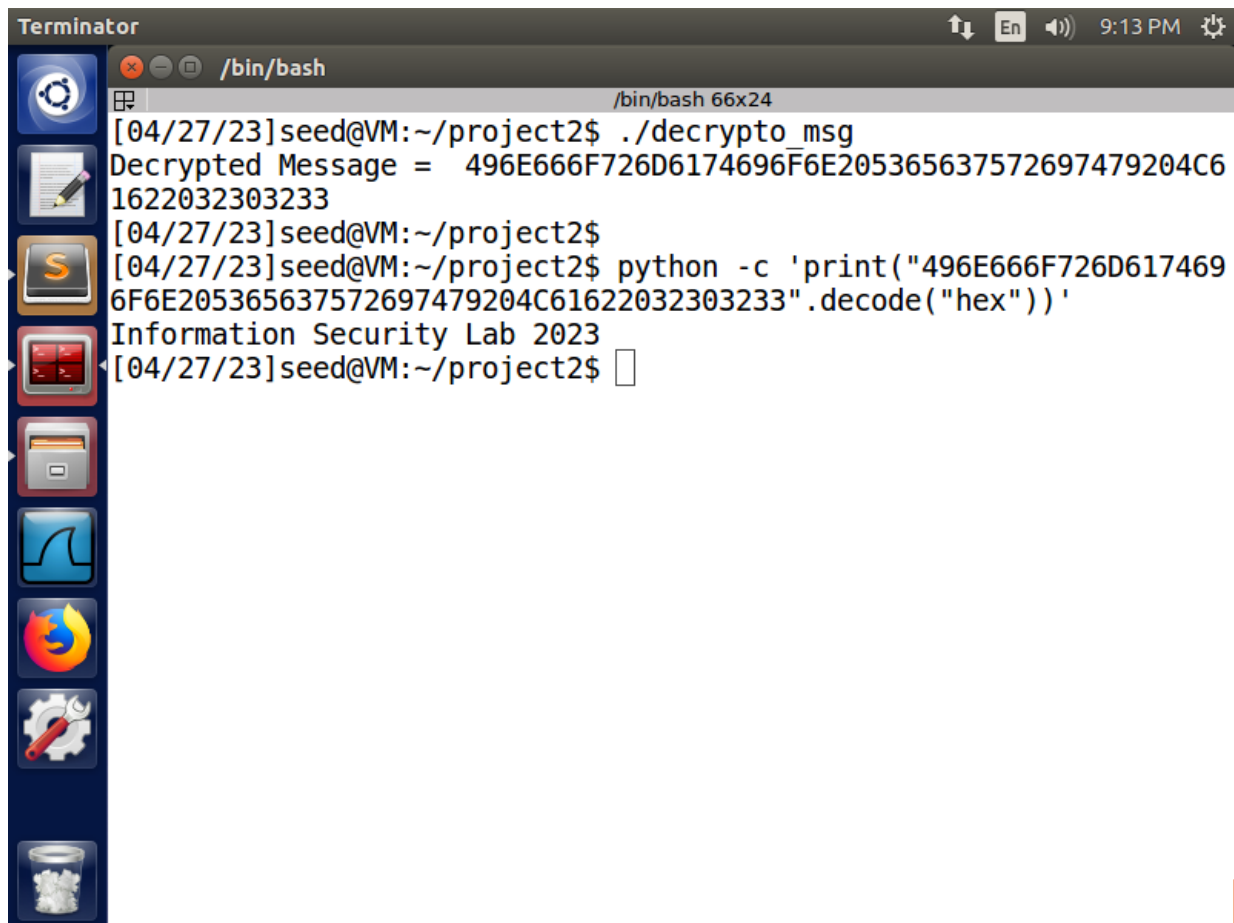
```
python -c  
'print ("4d79204e616d65".decode ("496E666F726D6174696F6E205365637572697479204C61622032303233")) '
```

3.2 Αποτελέσματα



```
Terminator                                     9:13 PM
/bin/bash                                     /bin/bash 66x24
[04/27/23]seed@VM:~/project2$ gcc decrypto_msg.c -o decrypto_msg -lcrypto
[04/27/23]seed@VM:~/project2$ ls -l decrypto_msg
-rwxrwxr-x 1 seed seed 7696 Apr 27 21:13 decrypto_msg
[04/27/23]seed@VM:~/project2$ ./decrypto_msg
Decrypted Message = 496E666F726D6174696F6E205365637572697479204C61622032303233
[04/27/23]seed@VM:~/project2$
```

Εικόνα 3.2.1. Μεταγλώττιση και εκτέλεση του προγράμματος `decrypto_msg.c`



The image shows a screenshot of a Terminator terminal window. The title bar reads 'Terminator' and includes standard window controls and system status (9:13 PM). The terminal prompt is `/bin/bash`. The user runs the command `./decrypto msg`, which outputs the decrypted message: `Decrypted Message = 496E666F726D6174696F6E205365637572697479204C61622032303233`. The user then runs a Python command to decode the hex string: `python -c 'print("496E666F726D6174696F6E205365637572697479204C61622032303233".decode("hex"))'`, which outputs `Information Security Lab 2023`. The terminal window has a sidebar with various application icons.

```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ ./decrypto msg
Decrypted Message = 496E666F726D6174696F6E205365637572697479204C61622032303233
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ python -c 'print("496E666F726D6174696F6E205365637572697479204C61622032303233".decode("hex"))'
Information Security Lab 2023
[04/27/23]seed@VM:~/project2$
```

Εικόνα 3.2.2. Επαλήθευση της αποκρυπτογράφησης του μηνύματος

3.3 Παρατηρήσεις

Η αποκρυπτογράφηση του μηνύματος υπολογίστηκε από τον μαθηματικό τύπο $D = C^d \bmod N$. Το μήνυμα «Information Security Lab 2023» επαληθεύει την επιτυχημένη αποκρυπτογράφηση του μηνύματος.

4. Υπογραφή μηνύματος

4.1 Ενέργειες

Στην δραστηριότητα αυτή δημιουργήσαμε την ψηφιακή υπογραφή ενός μηνύματος χάρις στο πρόγραμμα `sign_msg.c`. Το μήνυμα είναι «Hello World» και αρχικά το κωδικοποιήσαμε σε δεκαεξαδική μορφή με το κατάλληλο python script

Το `sign_msg.c` (Εικόνα 4.1.1) δημιουργεί την ψηφιακή υπογραφή του μηνύματος υλοποιώντας μαθηματικές πράξεις με μεγάλους αριθμούς χάρις στην βιβλιοθήκη συναρτήσεων BN που χειρίζεται αριθμούς που ξεφεύγουν από τα όρια της αρχιτεκτονικής υπολογιστών. Πιο αναλυτικά:

στη γραμμή 24 αρχικοποιεί το δημόσιο κλειδί στην δομή BN, e

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
BN_hex2bn(&e, "0D88C3");
```

στη γραμμή 29 αρχικοποιεί το modulo στην δομή BN, N

```
BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
```

στη γραμμή 34 αρχικοποιεί το ιδιωτικό κλειδί στην δομή BN, d

```
BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
```

στη γραμμή 38 αρχικοποιεί το κρυπτογραφημένο μήνυμα, κωδικοποιημένο σε δεκαεξαδική μορφή στην δομή BN, M1

```
BN_hex2bn(&M1, "48616C6C6F20576F726C64");
```

στη γραμμή 43 υπολογίζει την μαθηματική πράξη $M1^d \bmod N$ και εκχωρεί το αποτέλεσμα στη δομή BN, M2, δηλαδή, υπολογίζει την ψηφιακή υπογραφή του μηνύματος

```
BN_mod_exp(S, M1, d, N, ctx);
```

στη γραμμή 48 υπολογίζει την μαθηματική πράξη $S^e \bmod N$ και εκχωρεί το αποτέλεσμα στη δομή BN, M2, δηλαδή, αποκρυπτογραφεί την ψηφιακή υπογραφή του μηνύματος που είναι κωδικοποιημένο σε δεκαεξαδική μορφή

```
BN_mod_exp(M2, S, e, N, ctx);
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
bn_sample.c x private_key.c x crypto_msg.c x decrypto_msg.c x sign_msg.c x
1  #include <stdio.h>
2  #include <openssl/bn.h>
3
4  #define NBITS 256
5
6  void printBN(char *msg, BIGNUM *a);
7
8  int main(int argc, char *argv[])
9  {
10     /*
11      * Δημιουργία των δομών τύπου BN
12      */
13     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
14     BIGNUM *M1 = BN_new();       // Μήνυμα
15     BIGNUM *S = BN_new();       // Υπογραφή μηνύματος
16     BIGNUM *M2 = BN_new();       // Επαλήθευση υπογραφής
17     BIGNUM *e = BN_new();       // Δημόσιο κλειδί (e, N)
18     BIGNUM *N = BN_new();       // Modulo N
19     BIGNUM *d = BN_new();       // Ιδιωτικό κλειδί (d, N)
20
21     /*
22      * Αρχικοποίηση του δημόσιου κλειδιού (e, N) που χρησιμοποιήθηκε στην
23      * δραστηριότητα 3 : Αποκρυπτογράφηση μηνύματος
24      */
25     BN_hex2bn(&e, "0D88C3");
26
27     /*
28      * Αρχικοποίηση του modulo N που υπολογίστηκε στην
29      * δραστηριότητα 1 : Δημιουργία ιδιωτικού κλειδιού
30      */
31     BN_hex2bn(&N, "E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1");
32
33     /*
34      * Αρχικοποίηση του δημόσιου κλειδιού (d, N) που υπολογίστηκε στην
35      * δραστηριότητα 3 : Αποκρυπτογράφηση μηνύματος
36      */
37     BN_hex2bn(&d, "3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB");
38
39     /*
40      * Αρχικοποίηση του μηνύματος "Hello World"
41      */
42     BN_hex2bn(&M1, "48616C6C6F20576F726C64");
43
44     /*
45      * Υπολογισμός της ψηφιακής υπογραφής του μηνύματος με τον τύπο
46      *  $S = M1^d \mod N$ 
47      */
48     BN_mod_exp(S, M1, d, N, ctx);
49
50     /*
51      * Υπολογισμός της "αποκρυπτογράφησης" της ψηφιακής υπογραφής του μηνύματος
52      * με τον τύπο  $M2 = S^e \mod N$ 
53      */
54     BN_mod_exp(M2, S, e, N, ctx);
55
56     /*
57      * Εκτύπωση της ψηφιακής υπογραφής του μηνύματος
58      */
59     printBN("Message's Signature = ", S);
60
61     /*
62      * Εκτύπωση του μηνύματος
63      */
64     printBN("Message \t = ", M1);
65
66     /*
67      * Εκτύπωση της "αποκρυπτογραφημένης" ψηφιακής υπογραφής του μηνύματος
68      */
69     printBN("Verified Signature = ", M2);
70
71     /*
72      * Αποδέσμευση των δομών τύπου BN από την μνήμη
73      */
74     BN_CTX_free(ctx);
75     BN_free(M1);
76     BN_free(S);
77     BN_free(M2);
78     BN_free(e);
79     BN_free(N);
80     BN_free(d);
81
82     return 0;
83 }
84
85 /*
86 * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
87 */
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
78 void printBN(char *msg, BIGNUM *a)
79 {
80     char *number_str = BN_bn2hex(a);
81     printf("%s %s\n", msg, number_str);
82     OPENSSL_free(number_str);
83 }
```

Εικόνα 4.1.1. Το πρόγραμμα sign_msg.c

Στη συνέχεια, μεταγλωττίσαμε τον κώδικα με την παράμετρο

```
-lcrypto
```

όπου συνδέει την βιβλιοθήκη συναρτήσεων χειρισμού μεγάλων αριθμών, BN της OpenSSL.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc sign_msg.c -o sign_msg -lcrypto
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο sign_msg.

Στη συνέχεια, τρέξαμε το πρόγραμμα sign_msg με την εντολή :

```
./sign_msg
```

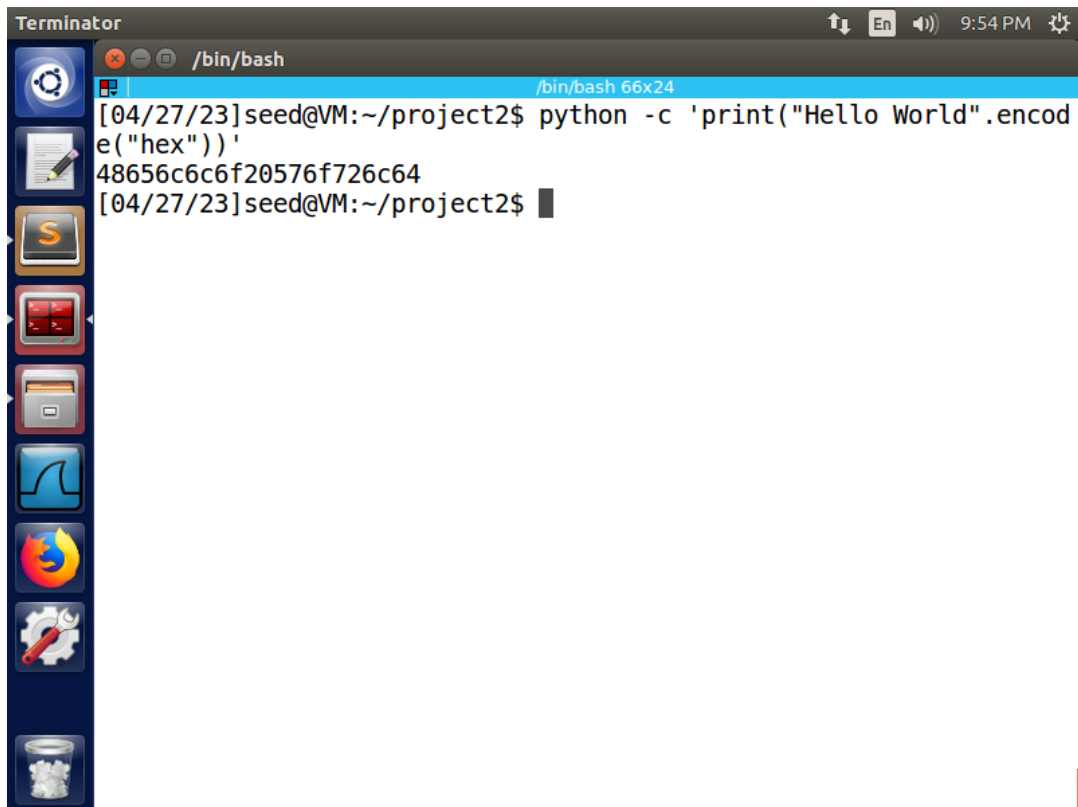
Τέλος, για να επαληθεύσουμε ότι το πρόγραμμα sign_msg.c αποκρυπτογραφεί την ψηφιακή υπογραφή του μηνύματος που είναι κρυπτογραφημένο και κωδικοποιημένο σε δεκαεξαδική μορφή, εκτελέσαμε ένα script της python, όπου αποκωδικοποιεί την συμβολοσειρά από δεκαεξαδική μορφή σε αναγνώσιμη μορφή.

Στη συνέχεια, αλλάξαμε το μήνυμα κατά ένα γράμμα σε «Hallo World» και το κωδικοποιήσαμε εκ νέου σε δεκαεξαδική μορφή

Στη συνέχεια, τρέξαμε το πρόγραμμα sign_msg (χωρίς να αλλάξουμε το d, e και N) με την εντολή :

```
./sign_msg
```

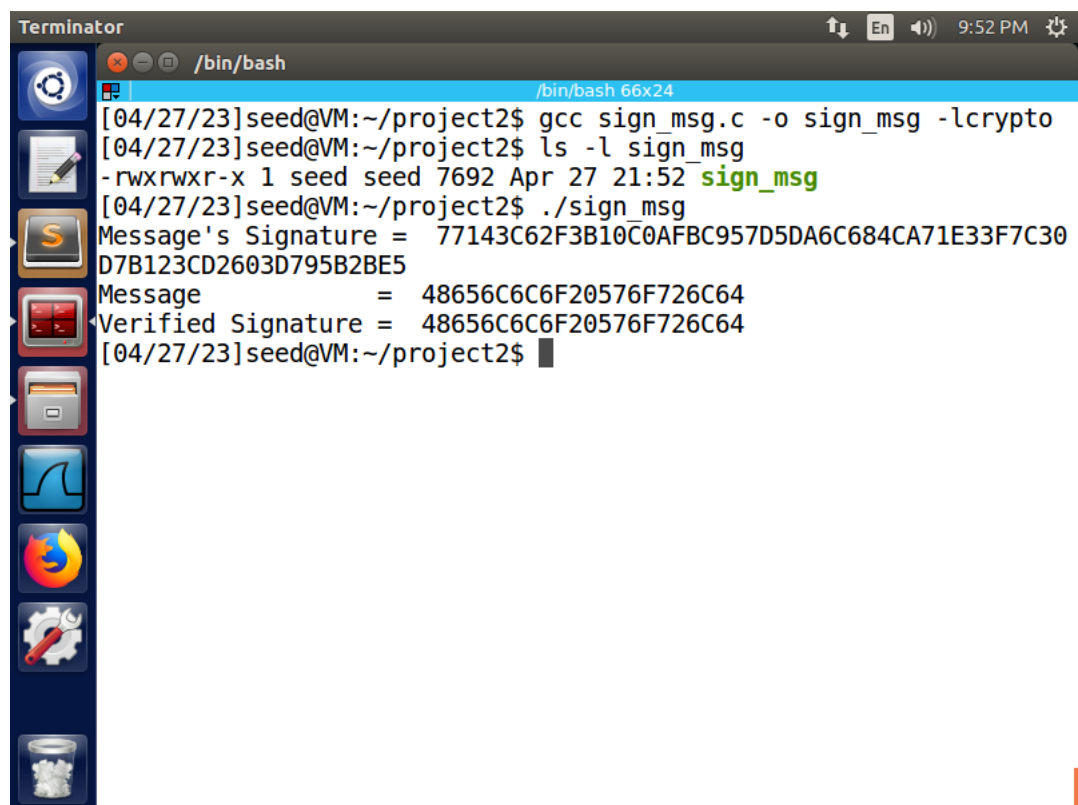
4.2 Αποτελέσματα



The screenshot shows a Terminator terminal window with a dark theme. The title bar includes the word "Terminator" and system icons for volume, network, and time (9:54 PM). The terminal window has a blue header bar with the text "/bin/bash" and "/bin/bash 66x24". The main area shows a command prompt [04/27/23]seed@VM:~/project2\$ followed by the command `python -c 'print("Hello World".encode("hex"))'`. The output is the hexadecimal string `48656c6c6f20576f726c64`. The prompt then shows [04/27/23]seed@VM:~/project2\$ followed by a cursor.

```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Hello World".encode("hex"))'
48656c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
```

Εικόνα 4.2.1. Κωδικοποίηση του μηνύματος «Hello World» σε δεκαεξαδική μορφή

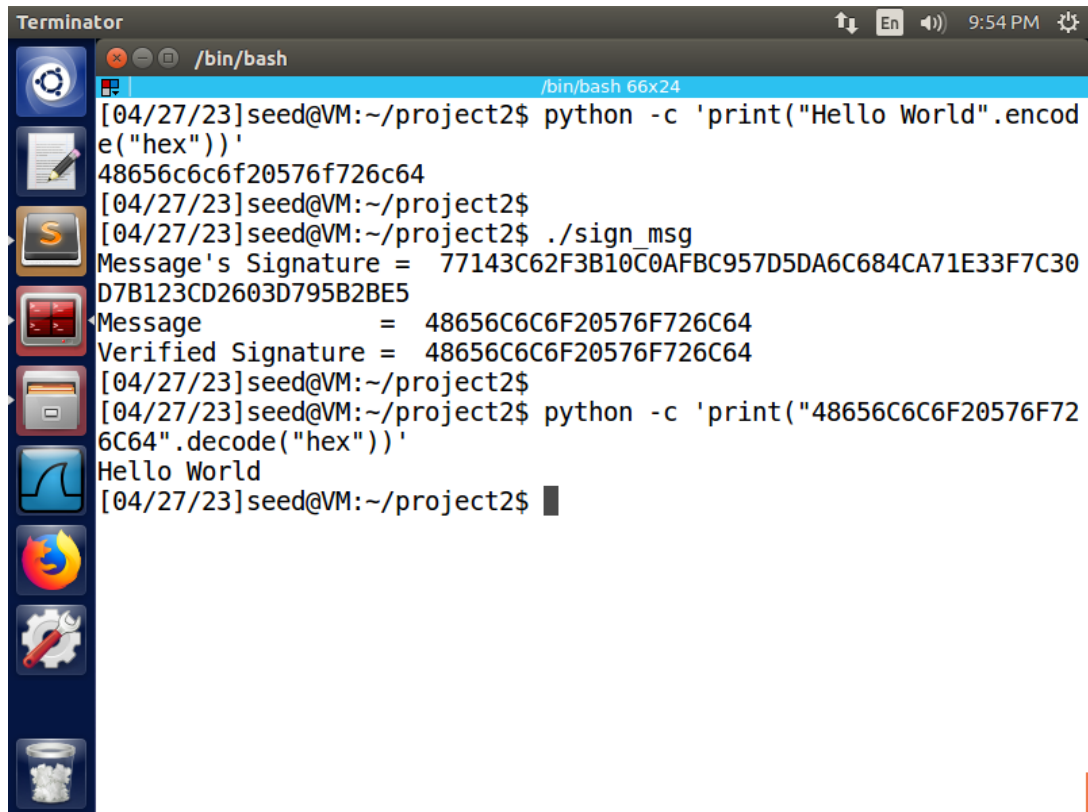


The screenshot shows a Terminator terminal window with a dark theme. The title bar includes the word "Terminator" and system icons for volume, network, and time (9:52 PM). The terminal window has a blue header bar with the text "/bin/bash" and "/bin/bash 66x24". The main area shows a command prompt [04/27/23]seed@VM:~/project2\$ followed by the command `gcc sign_msg.c -o sign_msg -lcrypto`. The next prompt shows [04/27/23]seed@VM:~/project2\$ followed by `ls -l sign_msg`, which outputs `-rwxrwxr-x 1 seed seed 7692 Apr 27 21:52 sign_msg`. The next prompt shows [04/27/23]seed@VM:~/project2\$ followed by `./sign_msg`, which outputs `Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30D7B123CD2603D795B2BE5`, `Message = 48656c6c6f20576f726c64`, and `Verified Signature = 48656c6c6f20576f726c64`. The prompt then shows [04/27/23]seed@VM:~/project2\$ followed by a cursor.

```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ gcc sign_msg.c -o sign_msg -lcrypto
[04/27/23]seed@VM:~/project2$ ls -l sign_msg
-rwxrwxr-x 1 seed seed 7692 Apr 27 21:52 sign_msg
[04/27/23]seed@VM:~/project2$ ./sign_msg
Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30D7B123CD2603D795B2BE5
Message = 48656c6c6f20576f726c64
Verified Signature = 48656c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
```

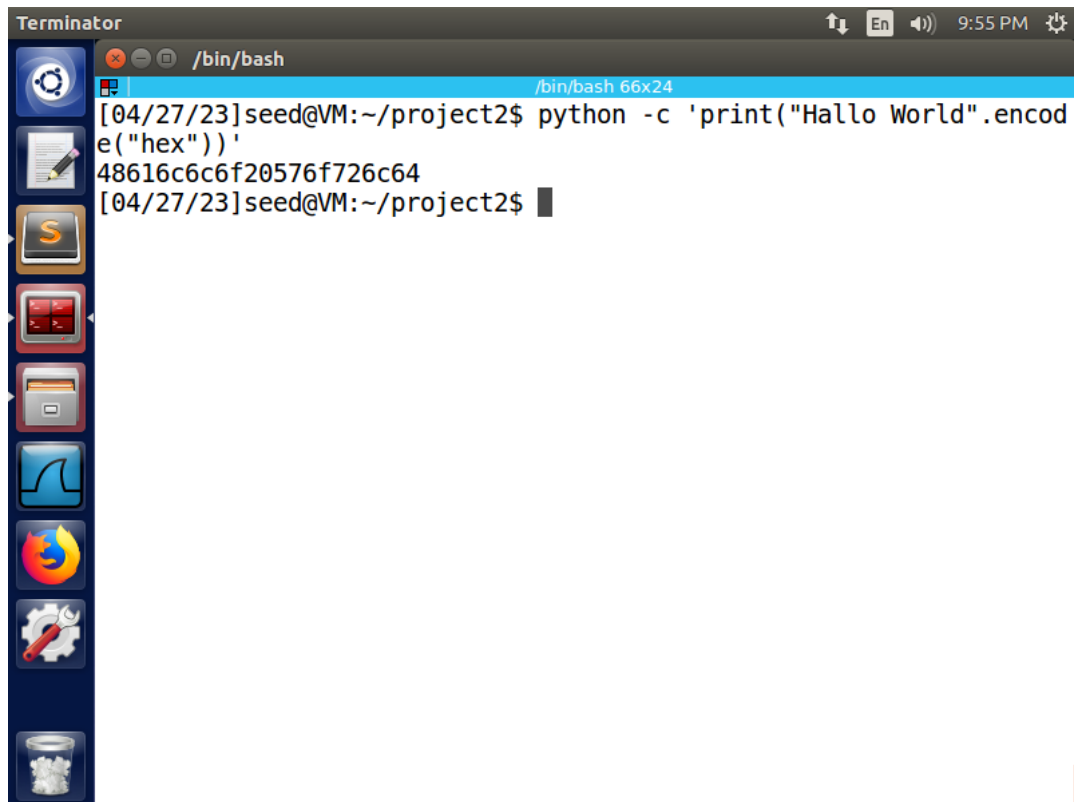
Εικόνα 4.2.2. Μεταγλώττιση και εκτέλεση του προγράμματος `sign_msg.c`

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



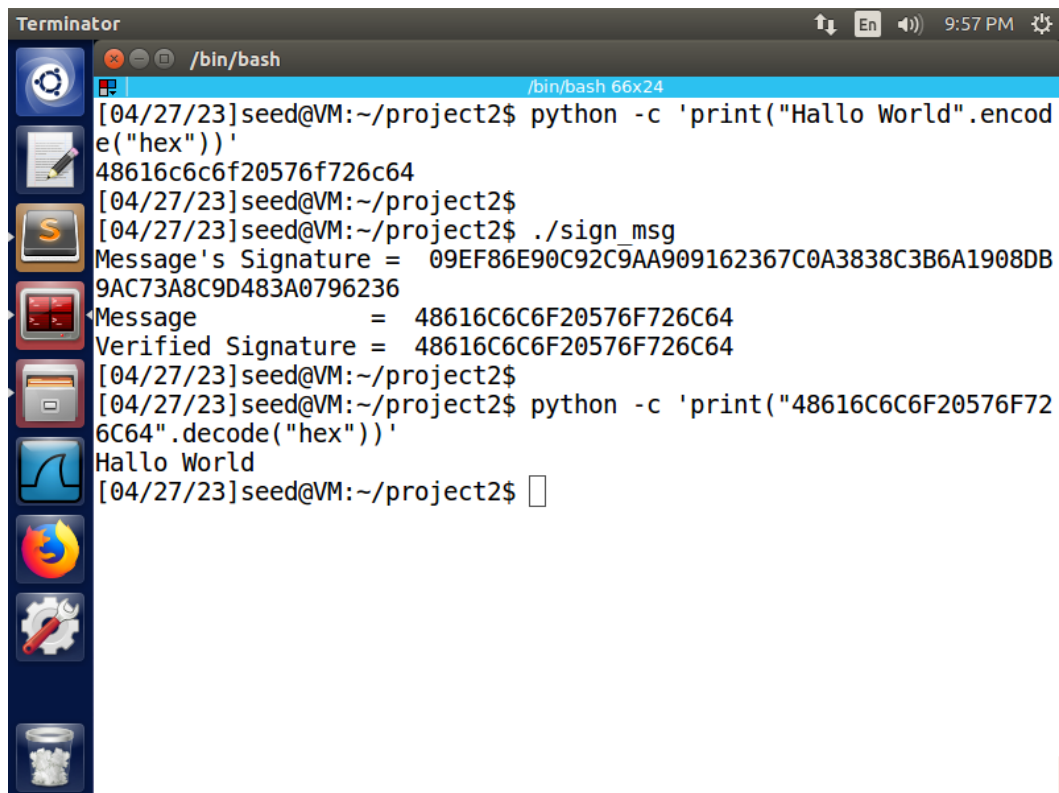
```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Hello World".encode("hex"))'
48656c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ ./sign_msg
Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30
D7B123CD2603D795B2BE5
Message = 48656c6c6f20576f726c64
Verified Signature = 48656c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ python -c 'print("48656c6c6f20576f726c64".decode("hex"))'
Hello World
[04/27/23]seed@VM:~/project2$
```

Εικόνα 4.2.3. Επαλήθευση της αποκρυπτογράφησης της ψηφιακής υπογραφής του μηνύματος «Hello World»



```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Hallo World".encode("hex"))'
48616c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
```

Εικόνα 4.2.4. Κωδικοποίηση του μηνύματος «Hallo World» σε δεκαεξαδική μορφή



```
Terminator
/bin/bash
/bin/bash 66x24
[04/27/23]seed@VM:~/project2$ python -c 'print("Hallo World".encode("hex"))'
48616c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ ./sign_msg
Message's Signature = 09EF86E90C92C9AA909162367C0A3838C3B6A1908DB
9AC73A8C9D483A0796236
Message = 48616c6c6f20576f726c64
Verified Signature = 48616c6c6f20576f726c64
[04/27/23]seed@VM:~/project2$
[04/27/23]seed@VM:~/project2$ python -c 'print("48616c6c6f20576f72
6c64".decode("hex"))'
Hallo World
[04/27/23]seed@VM:~/project2$
```

Εικόνα 4.2.5. Επαλήθευση της αποκρυπτογράφησης της ψηφιακής υπογραφής του μηνύματος «Hallo World»

4.3 Παρατηρήσεις

Οι ψηφιακές υπογραφές των μηνυμάτων υπολογίστηκαν από τον μαθηματικό τύπο $S = M1^d \bmod N$. Οι αποκρυπτογραφήσεις των ψηφιακών υπογραφών υπολογίστηκαν από τον μαθηματικό τύπο $M2 = S^e \bmod N$.

```
Message's Signature = 77143C62F3B10C0AFBC957D5DA6C684CA71E33F7C30
D7B123CD2603D795B2BE5
```

Εικόνα 4.3.1. Η ψηφιακή υπογραφή του μηνύματος «Hello World»

```
Message's Signature = 09EF86E90C92C9AA909162367C0A3838C3B6A1908DB
9AC73A8C9D483A0796236
```

Εικόνα 4.3.2. Η ψηφιακή υπογραφή του μηνύματος «Hallo World»

Παρόλο που η διαφορά των δύο μηνυμάτων είναι ένα γράμμα (ένα bit), οι υπογραφές τους είναι διαφορετικές, λόγω του τύπου υπολογισμού της ψηφιακής υπογραφής $S = M1^d \bmod N$. Το $M1$ αλλάζει πολύ ελάχιστα σε αντίθεση με την δύναμη $M1^d$ που αλλάζει κατά πολύ.

5. Επαλήθευση Υπογραφής

5.1 Ενέργειες

Στην δραστηριότητα αυτή επαληθεύσαμε την ψηφιακή υπογραφή ενός μηνύματος χάρις στο πρόγραμμα `verify_sign_msg.c`.

Περίπτωση Α

Το μήνυμα είναι «Launch a missile» και αρχικά το κωδικοποιήσαμε σε δεκαεξαδική μορφή με το κατάλληλο python script. Πομπός είναι ο Bob και δέκτης είναι η Alice

Το `verify_sign_msg.c` (Εικόνα 5.1.1) επαληθεύει την ψηφιακή υπογραφή του μηνύματος υλοποιώντας μαθηματικές πράξεις με μεγάλους αριθμούς χάρις στην βιβλιοθήκη συναρτήσεων BN που χειρίζεται αριθμούς που ξεφεύγουν από τα όρια της αρχιτεκτονικής υπολογιστών. Πιο αναλυτικά:

στη γραμμή 23 αρχικοποιεί το δημόσιο κλειδί στην δομή BN, `e`

```
BN_hex2bn(&e, "010001");
```

στη γραμμή 27 αρχικοποιεί το modulo στην δομή BN, `N`

```
BN_hex2bn(&N, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
```

στη γραμμή 31 αρχικοποιεί τη ψηφιακή υπογραφή του μηνύματος στην δομή BN, `S`

```
BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F");
```

στη γραμμή 35 αρχικοποιεί το κρυπτογραφημένο μήνυμα, κωδικοποιημένο σε δεκαεξαδική μορφή στην δομή BN, `M1`

```
BN_hex2bn(&M1, "4C61756E63682061206D697373696C652E");
```

στη γραμμή 40 υπολογίζει την μαθηματική πράξη $S^e \bmod N$ και εκχωρεί το αποτέλεσμα στη δομή BN, `M2`, δηλαδή, υπολογίζει την αποκρυπτογράφηση της ψηφιακής υπογραφής του μηνύματος

```
BN_mod_exp(M2, S, e, N, ctx);
```

στις γραμμές 56-64 συγκρίνει το μήνυμα με την επαλήθευση της υπογραφής του μηνύματος

```
areEqual = BN_cmp(M1, M2);  
switch (areEqual)  
{  
    case 0 :  
        printf("Message and Verified Signature are equal.\n");  
        break;  
    default :  
        printf("Message and Verified Signature are NOT equal.\n");  
}
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
br private_key.c x crypto_msg.c x decrypto_msg.c x sign_msg.c x verify_sign.c x
1 #include <stdio.h>
2 #include <openssl/bn.h>
3
4 #define NBITS 256
5
6 void printBN(char *msg, BIGNUM *a);
7
8 int main(int argc, char *argv[])
9 {
10     int areEqual;
11     /*
12      * Δημιουργία των δομών τύπου BN
13      */
14     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
15     BIGNUM *M1 = BN_new(); // Μήνυμα
16     BIGNUM *M2 = BN_new(); // Επαλήθευση υπογραφής
17     BIGNUM *S = BN_new(); // Υπογραφή μηνύματος
18     BIGNUM *e = BN_new(); // Δημόσιο κλειδί (e, N)
19     BIGNUM *N = BN_new(); // Modulo N
20     /*
21      * Αρχικοποίηση του δημόσιου κλειδιού (e, N)
22      */
23     BN_hex2bn(&e, "010001");
24     /*
25      * Αρχικοποίηση του modulo N
26      */
27     BN_hex2bn(&N, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
28     /*
29      * Αρχικοποίηση της υπογραφής του μηνύματος
30      */
31     BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F");
32     /*
33      * Αρχικοποίηση του μηνύματος "Launch a missile."
34      */
35     BN_hex2bn(&M1, "4C61756E63682061206D6973737696C652E");
36     /*
37      * Υπολογισμός της "αποκρυπτογράφησης" της ψηφιακής υπογραφής του μηνύματος
38      * με τον τύπο  $M2 = S^e \text{ mod } N$ 
39      */
40     BN_mod_exp(M2, S, e, N, ctx);
41     /*
42      * Εκτύπωση της ψηφιακής υπογραφής του μηνύματος
43      */
44     printBN("Message's Signature =", S);
45     /*
46      * Εκτύπωση του μηνύματος
47      */
48     printBN("Message \t = ", M1);
49     /*
50      * Εκτύπωση της "αποκρυπτογραφημένης" ψηφιακής υπογραφής του μηνύματος
51      */
52     printBN("Verified Signature = ", M2);
53     /*
54      * Σύγκριση του μηνύματος με την επαλήθευση υπογραφής
55      */
56     areEqual = BN_cmp(M1, M2);
57     switch (areEqual)
58     {
59         case 0 :
60             printf("Message and Verified Signature are equal.\n");
61             break;
62         default :
63             printf("Message and Verified Signature are NOT equal.\n");
64     }
65     /*
66      * Αποδέσμευση των δομών τύπου BN από την μνήμη
67      */
68     BN_CTX_free(ctx);
69     BN_free(M1);
70     BN_free(S);
71     BN_free(M2);
72     BN_free(e);
73     BN_free(N);
74 }
```

```
75     return 0;
76 }
77
78 /*
79  * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
80  */
81 void printBN(char *msg, BIGNUM *a)
82 {
83     char *number_str = BN_bn2hex(a);
84     printf("%s %s\n", msg, number_str);
85     OPENSSL_free(number_str);
86 }
```

Εικόνα 5.1.1. Το πρόγραμμα `verify_sign_msg.c`

Στη συνέχεια, μεταγλωττίσαμε τον κώδικα με την παράμετρο

```
-lcrypto
```

όπου συνδέει την βιβλιοθήκη συναρτήσεων χειρισμού μεγάλων αριθμών, BN της OpenSSL.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc verify_sign_msg.c -o verify_sign_msg -lcrypto
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο `verify_sign_msg`.

Στη συνέχεια, τρέξαμε το πρόγραμμα `verify_sign_msg` με την εντολή :

```
./verify_sign_msg
```

Τέλος, για να επαληθεύσουμε ότι το πρόγραμμα `verify_sign_msg.c` αποκρυπτογραφεί την ψηφιακή υπογραφή του μηνύματος του Bob που είναι κρυπτογραφημένο και κωδικοποιημένο σε δεκαεξαδική μορφή, εκτελέσαμε ένα script της python, όπου αποκωδικοποιεί την συμβολοσειρά από δεκαεξαδική μορφή σε αναγνώσιμη μορφή. Η εντολή είναι :

Στη συνέχεια, αλλοιώσαμε την ψηφιακή υπογραφή του μηνύματος (Εικόνα 5.1.2) κατά ένα bit (από 2F σε 3F στο τέλος) και επαναλάβαμε την δραστηριότητα

Τρέξαμε το πρόγραμμα `verify_sign_msg` (με ίδιες τις άλλες τιμές `e`, `N`, `M1`) με την εντολή :

```
./verify_sign_msg
```

```
19
20 Αρχικοποίηση του δημόσιου κλειδιού (e, N)
21
22 BN_hex2bn(&e, "010001");
23
24 Αρχικοποίηση του modulo N
25
26 BN_hex2bn(&N, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
27
28 Αρχικοποίηση της υπογραφής του μηνύματος
29
30 BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB68E3F");
31
32 Αρχικοποίηση του μηνύματος "Launch a missile."
33
34 BN_hex2bn(&M1, "4C61756E63682061206D697373696C652E");
35
```

Εικόνα 5.1.2 Αλλοίωση της ψηφιακής υπογραφής του μηνύματος του Bob

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Περίπτωση Β

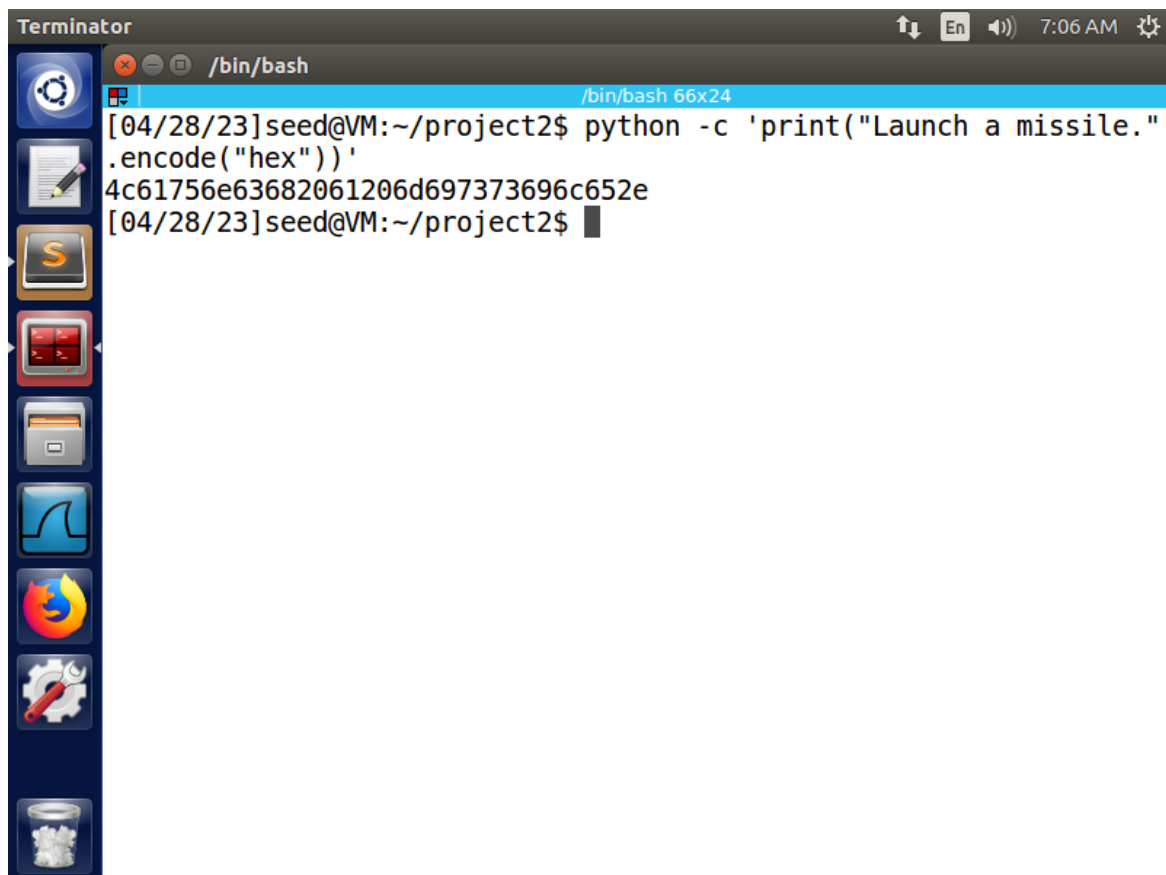
Σε αυτή την περίπτωση η Alice στέλνει στον Bob το μήνυμα «Please transfer me \$2000.Alice.» και επαναλάβαμε την διαδικασία (επαλήθευση υπογραφής) με διαφορές στις τιμές του e (δημόσιο κλειδί), του N (modulo) και του S (ψηφιακή υπογραφή) οι οποίες απεικονίζονται στην Εικόνα 5.1.3.

```
20
21 Αρχικοποίηση του δημόσιου κλειδιού ( $e$ ,  $N$ )
22
23 BN_hex2bn(&e, "010001");
24
25 Αρχικοποίηση του modulo  $N$ 
26
27 BN_hex2bn(&N, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDD3A4D0CB81629242FB1A5");
28
29 Αρχικοποίηση της υπογραφής του μηνύματος
30
31 BN_hex2bn(&S, "DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA6EDF3E95E9A7C6C7A320");
32
33 Αρχικοποίηση του μηνύματος "Please transfer me $2000.Alice."
34
35 BN_hex2bn(&M1, "506C65617365207472616E73666572206D652024323030302E416C6963652E");
36
```

Εικόνα 5.1.3. Οι τιμές για την επαλήθευση της υπογραφής του μηνύματος της Alice

5.2 Αποτελέσματα

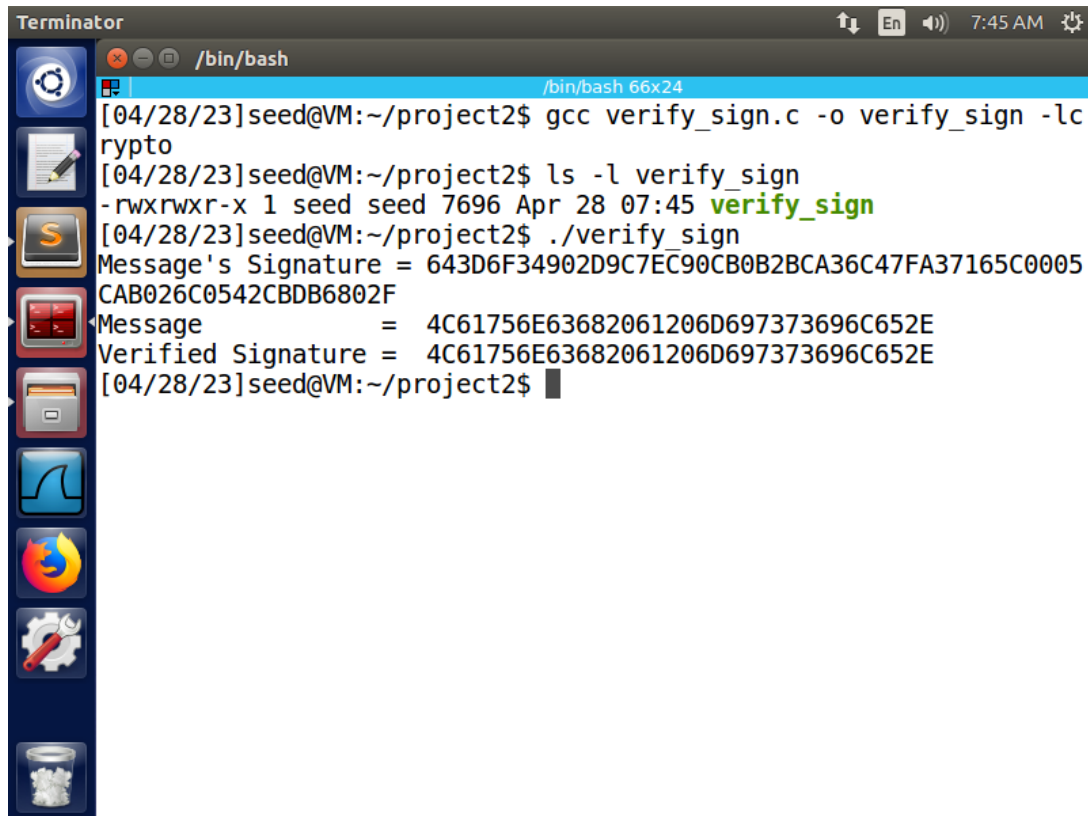
Περίπτωση Α



```
Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Launch a missile."
.encode("hex"))'
4c61756e63682061206d697373696c652e
[04/28/23]seed@VM:~/project2$
```

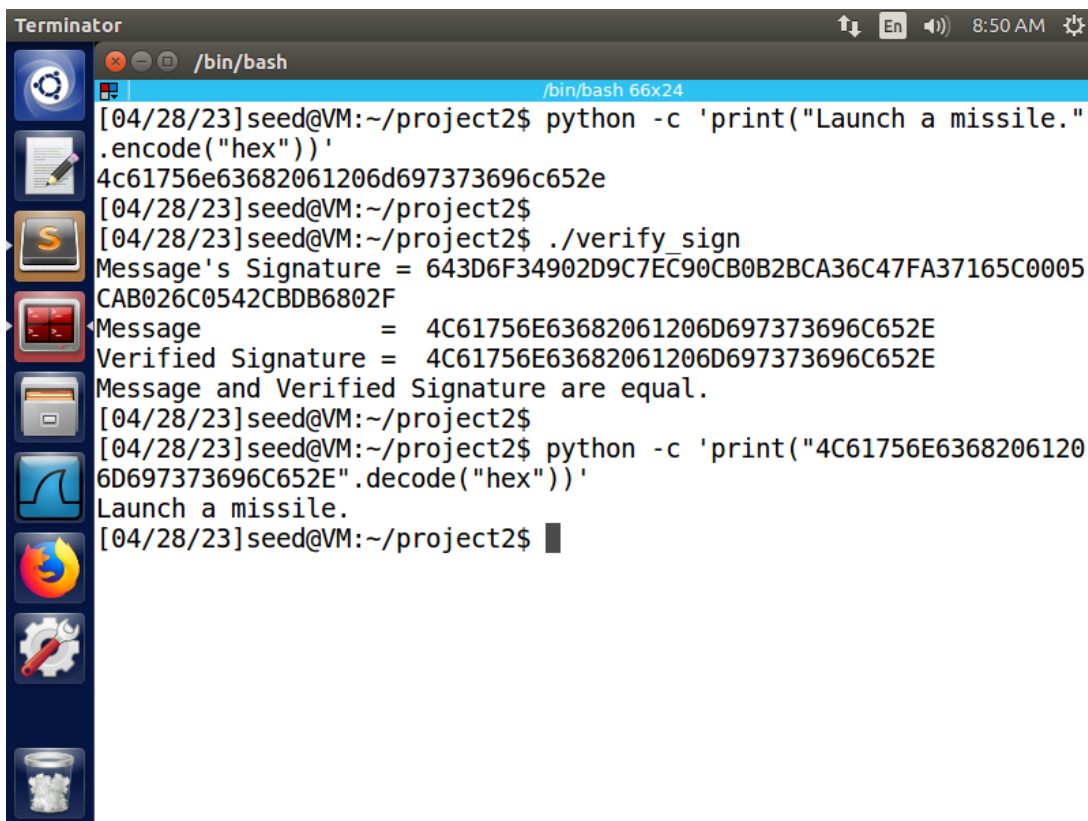
Εικόνα 5.2.1. Κωδικοποίηση του μηνύματος «Launch a missile.» σε δεκαεξαδική μορφή

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ gcc verify_sign.c -o verify_sign -lcrypto
[04/28/23]seed@VM:~/project2$ ls -l verify_sign
-rwxrwxr-x 1 seed seed 7696 Apr 28 07:45 verify_sign
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F
Message = 4C61756E63682061206D697373696C652E
Verified Signature = 4C61756E63682061206D697373696C652E
[04/28/23]seed@VM:~/project2$
```

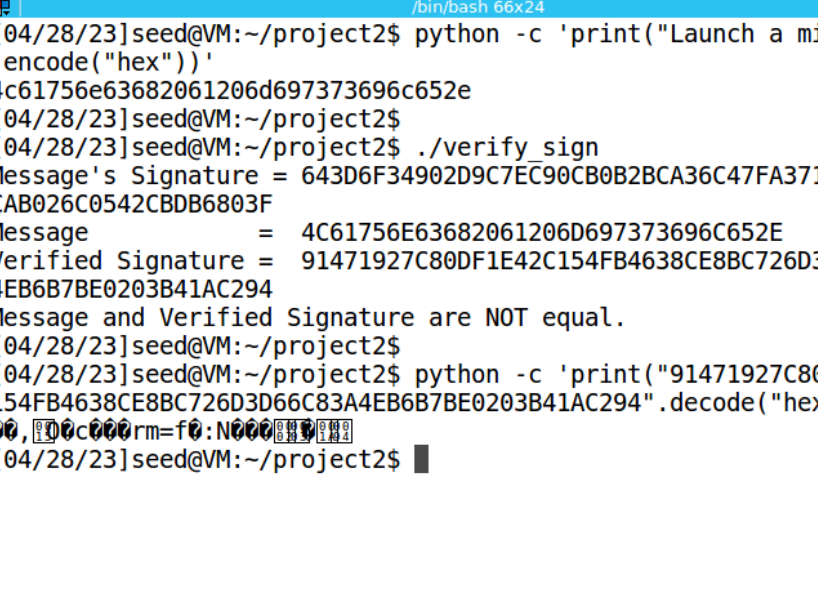
Εικόνα 5.2.2. Μεταγλώττιση και εκτέλεση του προγράμματος `verify_sign_msg.c` με το μήνυμα του Bob



```
Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Launch a missile."
.encode("hex"))'
4c61756e63682061206d697373696c652e
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F
Message = 4C61756E63682061206D697373696C652E
Verified Signature = 4C61756E63682061206D697373696C652E
Message and Verified Signature are equal.
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ python -c 'print("4C61756E6368206120
6D697373696C652E".decode("hex"))'
Launch a missile.
[04/28/23]seed@VM:~/project2$
```

Εικόνα 5.2.3. Επαλήθευση της αποκρυπτογράφησης της ψηφιακής υπογραφής του μηνύματος «Launch a missile.»

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Launch a missile."
.encode("hex"))'
4c61756e63682061206d697373696c652e
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = 643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005
CAB026C0542CBDB6803F
Message = 4C61756E63682061206D697373696C652E
Verified Signature = 91471927C80DF1E42C154FB4638CE8BC726D3D66C83A
4EB6B7BE0203B41AC294
Message and Verified Signature are NOT equal.
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ python -c 'print("91471927C80DF1E42C
154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294".decode("hex"))'
91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294
[04/28/23]seed@VM:~/project2$
```

Εικόνα 5.2.3. Αποκρυπτογράφηση της αλλοιωμένης ψηφιακής υπογραφής του μηνύματος «Launch a missile.»

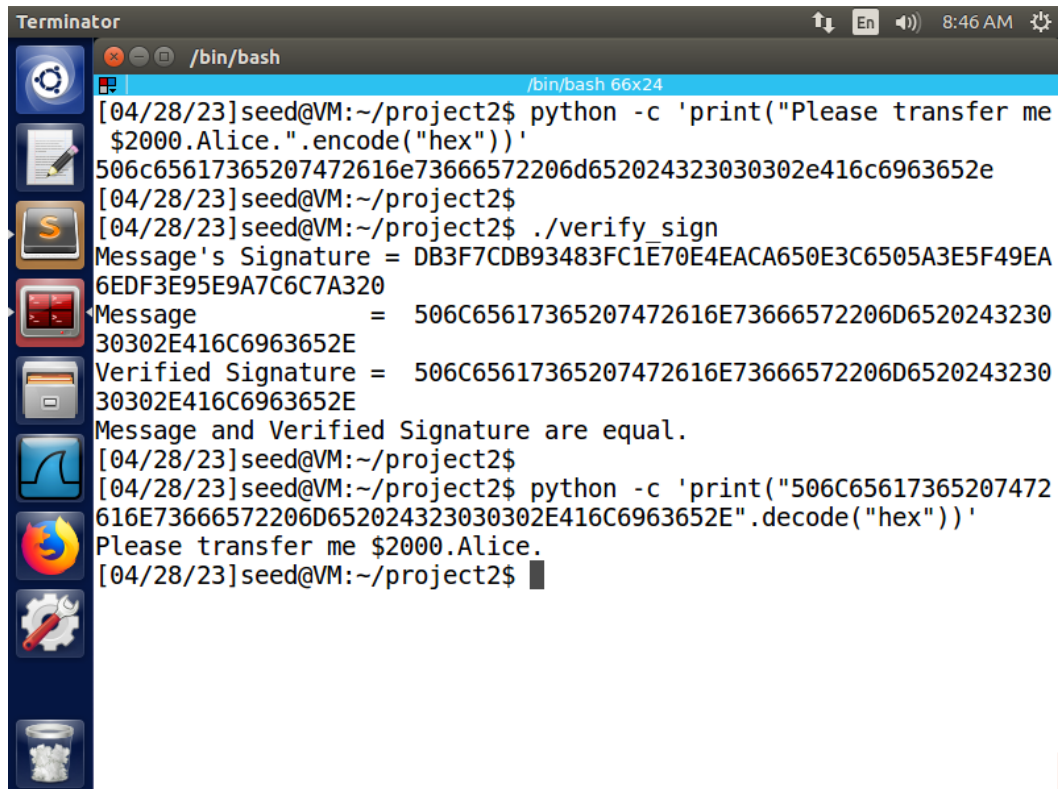
Περίπτωση Β

The screenshot shows a Kali Linux desktop environment. A terminal window is open, displaying the following command and output:

```
[04/28/23]seed@VM:~/project2$ python -c 'print("Please transfer me $2000.Alice.".encode("hex"))'
506c65617365207472616e73666572206d652024323030302e416c6963652e
[04/28/23]seed@VM:~/project2$
```

The desktop environment includes a dock on the left side with various application icons, such as a web browser, file manager, and terminal. The top of the window shows the title bar with the text "Terminator" and standard window controls.

Εικόνα 5.2.4. Κωδικοποίηση του μηνύματος «Please transfer me \$2000.Alice.» σε δεκαεξαδική μορφή



```
Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ python -c 'print("Please transfer me $2000.Alice.".encode("hex"))'
506c65617365207472616e73666572206d652024323030302e416c6963652e
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ ./verify_sign
Message's Signature = DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA6EDF3E95E9A7C6C7A320
Message = 506C65617365207472616E73666572206D652024323030302E416C6963652E
Verified Signature = 506C65617365207472616E73666572206D652024323030302E416C6963652E
Message and Verified Signature are equal.
[04/28/23]seed@VM:~/project2$
[04/28/23]seed@VM:~/project2$ python -c 'print("506C65617365207472616E73666572206D652024323030302E416C6963652E".decode("hex"))'
Please transfer me $2000.Alice.
[04/28/23]seed@VM:~/project2$
```

Εικόνα 5.2.5. Επαλήθευση της αποκρυπτογράφησης της ψηφιακής υπογραφής του μηνύματος «Please transfer me \$2000.Alice.»

5.3 Παρατηρήσεις

Οι αποκρυπτογραφήσεις των ψηφιακών υπογραφών υπολογίστηκαν από τον μαθηματικό τύπο $M2 = S^e \bmod N$. Στην περίπτωση A του μηνύματος του Bob (Launch a missile.), όπου αλλοιώσαμε κατά ένα bit την ψηφιακή υπογραφή του μηνύματος, παρατηρήσαμε κατά την επαλήθευση της υπογραφής, ότι το μήνυμα δεν ήταν το αρχικό. Αυτό αποδεικνύει ότι οι ψηφιακές υπογραφές των μηνυμάτων είναι μοναδικές.

6. Μη αυτόματη επαλήθευση πιστοποιητικού X.509

6.1 Ενέργειες

Στην δραστηριότητα αυτή επαληθεύσαμε την ψηφιακή υπογραφή ενός πιστοποιητικού X.509 χάρις στο πρόγραμμα `verify_sign_cert.c`. Επιλέξαμε τον web server `apachefriends.org` και για να επαληθεύσουμε την υπογραφή εκτελέσαμε τα παρακάτω βήματα.

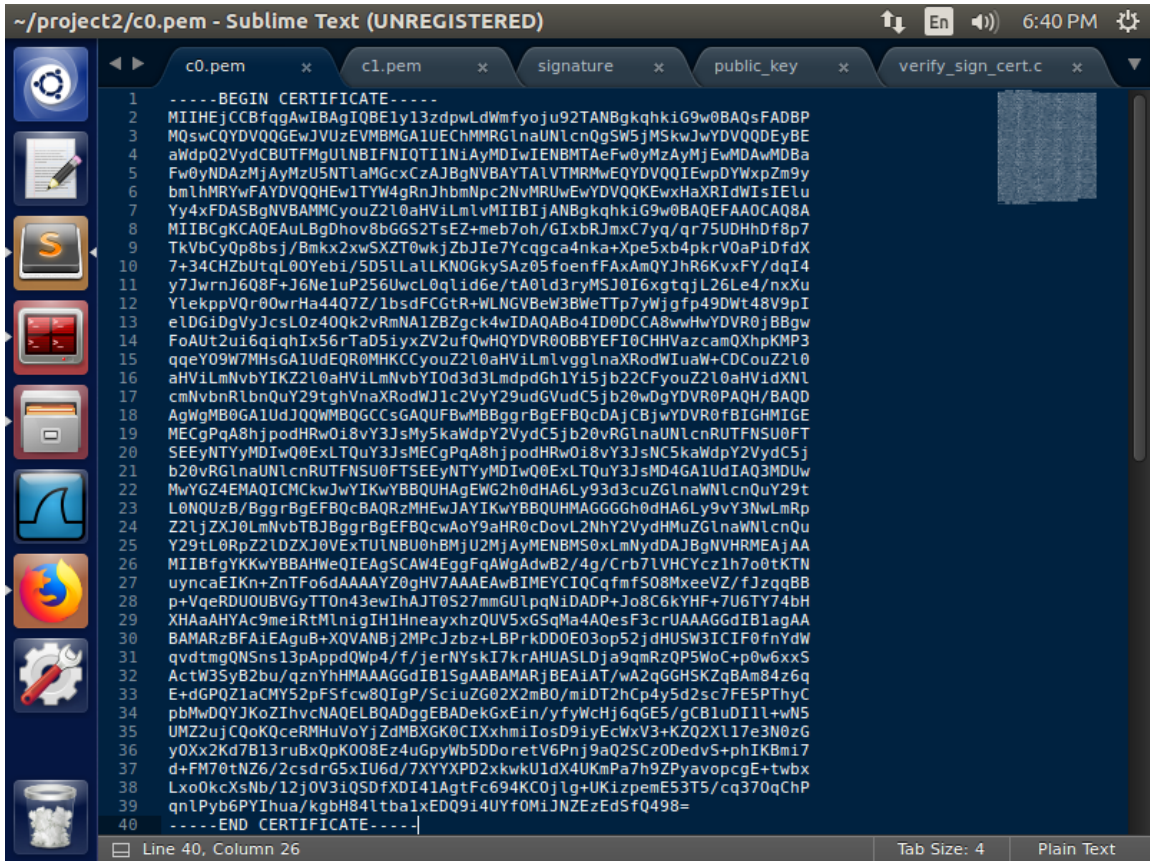
Βήμα 1

- Λήψη πιστοποιητικών με την εντολή

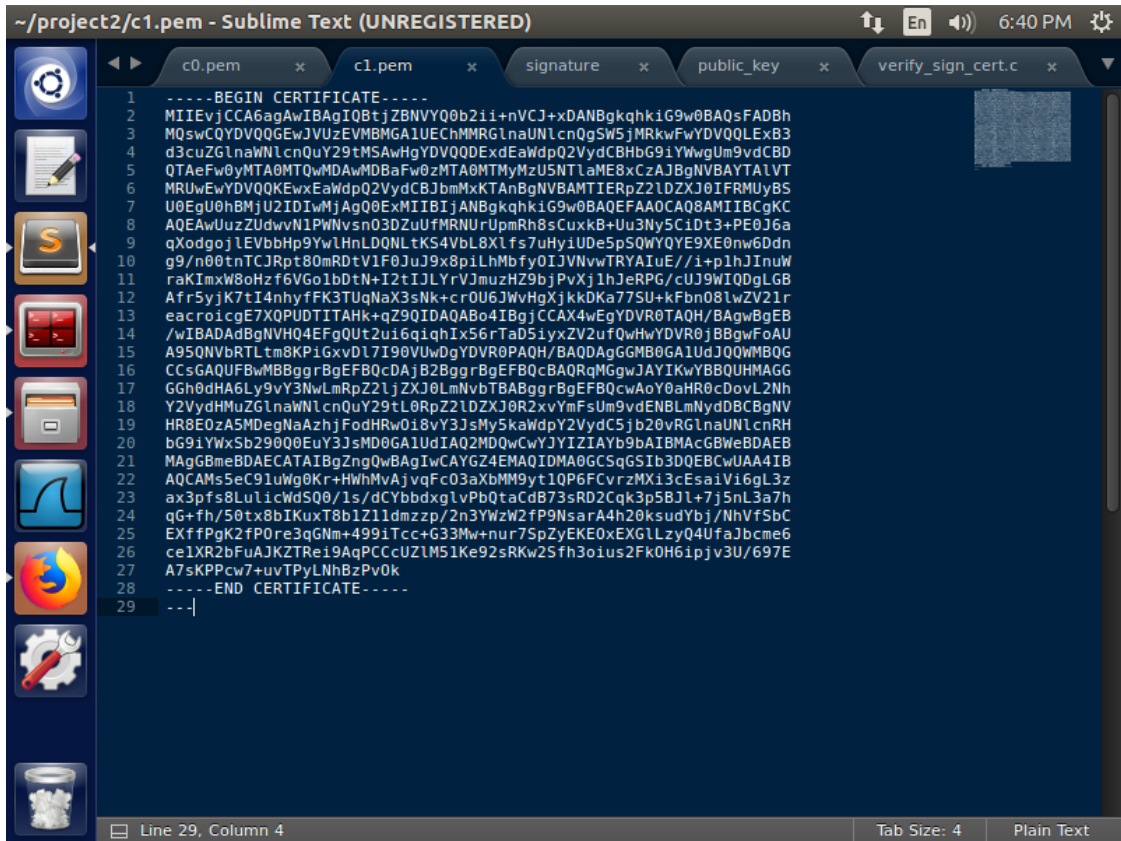
```
openssl s_client -connect www.apachefriends.org:443 -showcerts
```

- Αποθήκευση του πιστοποιητικού του server (πιστοποιητικό 0) στο αρχείο `c0.pem` (Εικόνα 6.1.1)
- Αποθήκευση του πιστοποιητικού του εκδότη (πιστοποιητικό 1) στο αρχείο `c1.pem` (Εικόνα 6.1.2)

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



Εικόνα 6.1.1. Πιστοποιητικό server



Εικόνα 6.1.2. Πιστοποιητικό εκδότη

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Βήμα 2

- Εξαγωγή modulo από το πιστοποιητικό του εκδότη με την εντολή

```
openssl x509 -in c1.pem -noout -modulus
```

- Εκτύπωση όλων των πεδίων για την εύρεση του e (δημοσίου κλειδιού) από το πιστοποιητικό του εκδότη με την εντολή

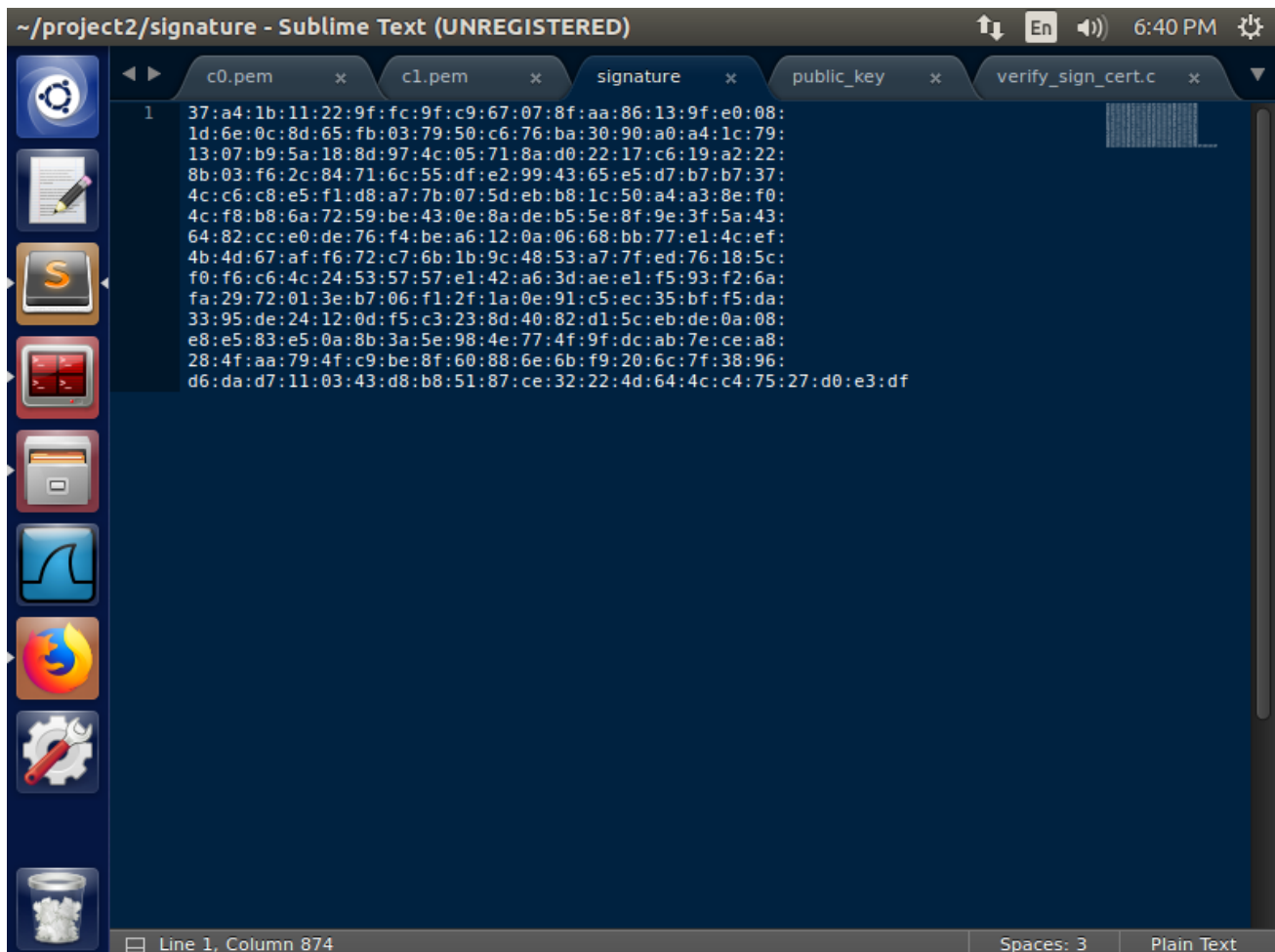
```
openssl x509 -in c1.pem -text -noout
```

Βήμα 3

- Εξαγωγή ψηφιακής υπογραφής S από το πιστοποιητικό του server

```
openssl x509 -in c0.pem -text -noout
```

- Αποθήκευση της υπογραφής στο αρχείο signature (Εικόνα 6.1.3)



Εικόνα 6.1.3. Η ψηφιακή υπογραφή από το πιστοποιητικό του server

- Αφαίρεση διαστημάτων και άνω τελειών από τα δεδομένα με την εντολή

```
cat signature | tr -d '[:space:]:'
```

Βήμα 4

- Εξαγωγή του σώματος του πιστοποιητικού του server με την εντολή

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
openssl asn1parse -i -in c0.pem
```

- Αποθήκευση του σώματος του πιστοποιητικού του server σε ένα δυαδικό αρχείο c0_body.bin με την εντολή

```
openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout
```

- Υπολογισμός του hash

```
sha256sum c0_body.bin
```

Βήμα 5

Αφού συλλέξαμε τις πληροφορίες, στο βήμα αυτό επαληθεύσαμε την ψηφιακή υπογραφή του πιστοποιητικού χάρις στο πρόγραμμα verify_sign_cert.c (Εικόνα 6.1.4). Πιο αναλυτικά :

Το verify_sign_msg.c (Εικόνα 5.1.1) επαληθεύει την ψηφιακή υπογραφή του μηνύματος υλοποιώντας μαθηματικές πράξεις με μεγάλους αριθμούς χάρις στην βιβλιοθήκη συναρτήσεων BN που χειρίζεται αριθμούς που ξεφεύγουν από τα όρια της αρχιτεκτονικής υπολογιστών. Πιο αναλυτικά:

στη γραμμή 23 αρχικοποιεί το δημόσιο κλειδί στην δομή BN, e

```
BN_hex2bn(&e, "10001");
```

στη γραμμή 27 αρχικοποιεί το modulo στην δομή BN, N

```
BN_hex2bn(&N, "c14bb3654770bcdd4f58dbec9cedc366e51f311354ad4a66461f2c0aec6407e52edcdcb90a20eddfc3c4d09e9aa97a1d8288e51156db1e9f58c251e72c340d2ed292e156cbf1795fb3bb87ca25037b9a52416610604f571349f0e8376783dfe7d34b674c2251a6df0e9910ed57517426e27dc7ca622e131b7f238825536fc13458008b84fff8bea75849227b96ada2889b15bca07cdfe951a8d5b0ed37e236b4824b62b5499aeecc767d6e33ef5e3d6125e44f1bf71427d58840380b18101faf9ca32bbb48e278727c52b74d4a8d697dec364f9cace53a256bc78178e490329aefb494fa415b9cef25c19576d6b79a72ba2272013b5d03d40d321300793ea99f5");
```

στη γραμμή 31 αρχικοποιεί τη ψηφιακή υπογραφή του πιστοποιητικού στην δομή BN, S

```
BN_hex2bn(&S, "37a41b11229ffc9fc967078faa86139fe0081d6e0c8d65fb037950c676ba3090a0a41c791307b95a188d974c05718ad02217c619a2228b03f62c84716c55dfe2994365e5d7b7b7374cc6c8e5f1d8a77b075debb81c50a4a38ef04cf8b86a7259be430e8adeb55e8f9e3f5a436482cce0de76f4bea6120a0668bb77e14cef4b4d67aff672c76b1b9c4853a77fed76185cf0f6c64c24535757e142a63daee1f593f26afa2972013eb706f12f1a0e91c5ec35bfff5da3395de24120df5c3238d4082d15cebde0a08e8e583e50a8b3a5e984e774f9fdcab7ecea8284faa794fc9be8f60886e6bf9206c7f3896d6dad7110343d8b85187ce32224d644cc47527d0e3df");
```

στη γραμμή 35 αρχικοποιεί το hash value, κωδικοποιημένο σε δεκαεξαδική μορφή στην δομή BN, H

```
BN_hex2bn(&H, "ed799631340493b633ba23ec5748dcae2ae1eef862fab54b369c044e2aad4a63");
```

στη γραμμή 40 υπολογίζει την μαθηματική πράξη $S^e \bmod N$ και εκχωρεί το αποτέλεσμα στη δομή BN, D, δηλαδή, υπολογίζει την αποκρυπτογράφηση της ψηφιακής υπογραφής του πιστοποιητικού

```
BN_mod_exp(D, S, e, N, ctx);
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
verify_sign_cert.c x
1 #include <stdio.h>
2 #include <openssl/bn.h>
3
4 #define NBITS 256
5
6 void printBN(char *msg, BIGNUM *a);
7
8 int main(int argc, char *argv[])
9 {
10     int areEqual;
11     /*
12      * Δημιουργία των δομών τύπου BN
13      */
14     BN_CTX *ctx = BN_CTX_new(); // Δομή απομνημόνευσης δομών τύπου BN
15     BIGNUM *D = BN_new(); // Μήνυμα
16     BIGNUM *H = BN_new(); // Επαλήθευση υπογραφής
17     BIGNUM *S = BN_new(); // Υπογραφή μηνύματος
18     BIGNUM *e = BN_new(); // Δημόσιο κλειδί (e, N)
19     BIGNUM *N = BN_new(); // Modulo N
20     /*
21      * Αρχικοποίηση του δημόσιου κλειδιού (e, N) από το πιστοποιητικό του εκδότη
22      */
23     BN_hex2bn(&e, "10001");
24     /*
25      * Αρχικοποίηση του modulo N από το πιστοποιητικό του εκδότη
26      */
27     BN_hex2bn(&N, "c14bb3654770bcdd4f58dbec9cedc366e51f311354ad4a66461f2c0aec6407e5");
28     /*
29      * Αρχικοποίηση της υπογραφής από το πιστοποιητικό του server
30      */
31     BN_hex2bn(&S, "37a41b11229ffc9fc967078faa86139fe0081d6e0c8d65fb037950c676ba3096");
32     /*
33      * Αρχικοποίηση του ψηφιακού αποτυπώματος H μέσω υπολογισμού από την συνάρτηση hash
34      */
35     BN_hex2bn(&H, "ed799631340493b633ba23ec5748dcae2ae1eef862fab54b369c044e2aad4a63");
36     /*
37      * Υπολογισμός της "αποκρυπτογράφησης" της ψηφιακής υπογραφής από
38      * το πιστοποιητικό του server με τον τύπο  $D = S^e \bmod N$ 
39      */
40     BN_mod_exp(D, S, e, N, ctx);
41     /*
42      * Εκτύπωση του ψηφιακού αποτυπώματος
43      */
44     printBN("Hash \t\t =", H);
45     /*
46      * Εκτύπωση της "αποκρυπτογραφημένης" ψηφιακής υπογραφής
47      */
48     printBN("Digest \t\t =", D);
49     /*
50      * Αποδέσμευση των δομών τύπου BN από την μνήμη
51      */
52     BN_CTX_free(ctx);
53     BN_free(D);
54     BN_free(H);
55     BN_free(S);
56     BN_free(e);
57     BN_free(N);
58
59     return 0;
60 }
61
62 /*
63 * Συνάρτηση που τυπώνει στο output, δομές τύπου BN
64 */
65 void printBN(char *msg, BIGNUM *a)
66 {
67     char *number_str = BN_bn2hex(a);
68     printf("%s %s\n\n", msg, number_str);
69     OPENSSL_free(number_str);
70 }
```

Εικόνα 6.1.5. Το πρόγραμμα verify_sing_cert.c

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Στη συνέχεια, μεταγλωττίσαμε τον κώδικα με την παράμετρο

```
-lcrypto
```

όπου συνδέει την βιβλιοθήκη συναρτήσεων χειρισμού μεγάλων αριθμών, BN της OpenSSL.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc verify_sign_cert.c -o verify_sign_cert -lcrypto
```

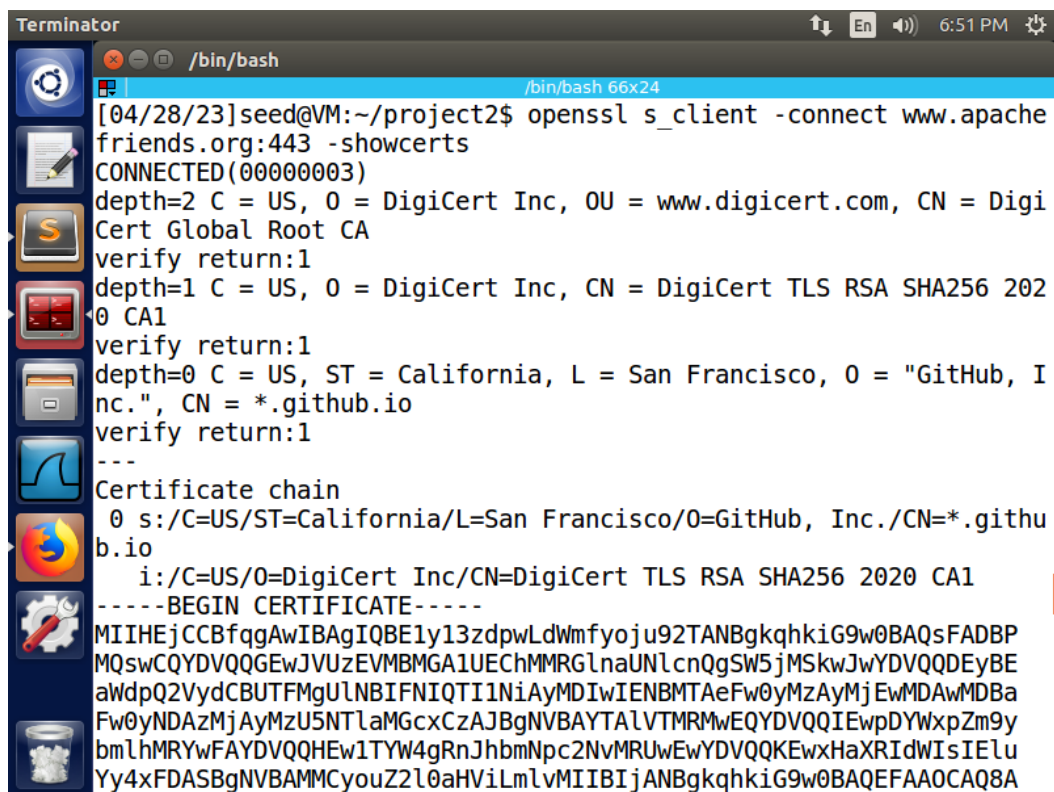
και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο verify_sign_cert.

Στη συνέχεια, τρέξαμε το πρόγραμμα verify_sign_msg με την εντολή :

```
./verify_sign_cert
```

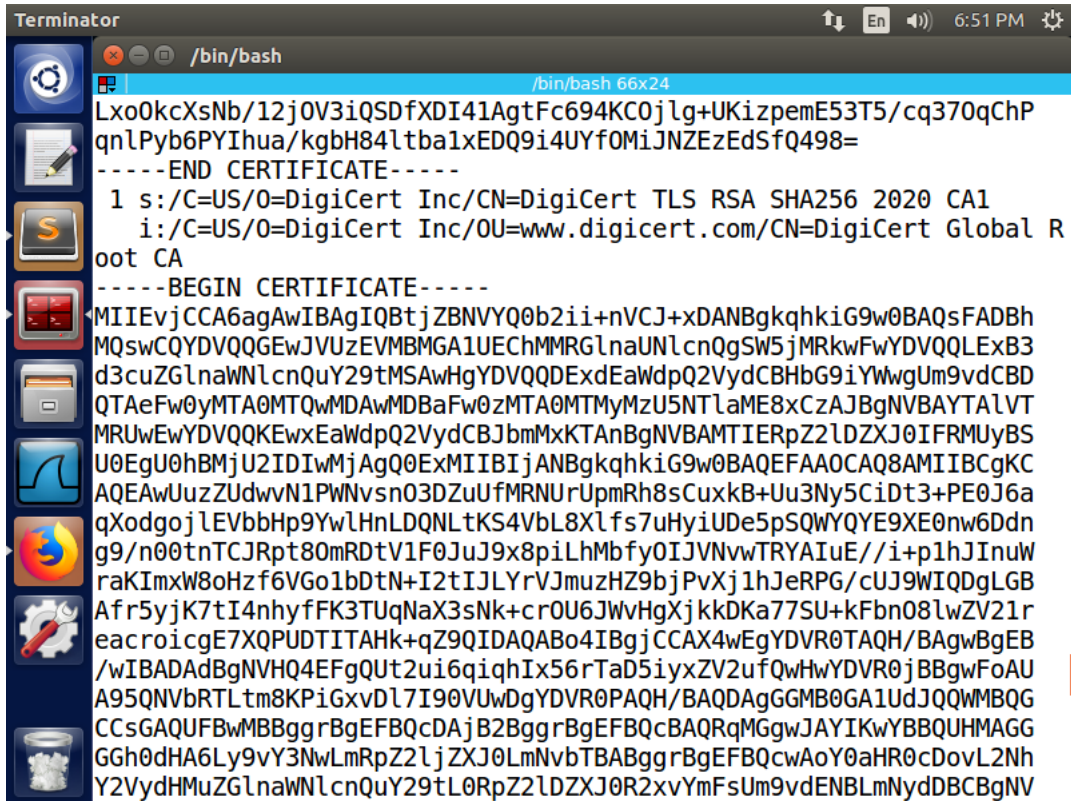
6.2 Αποτελέσματα

Βήμα 1



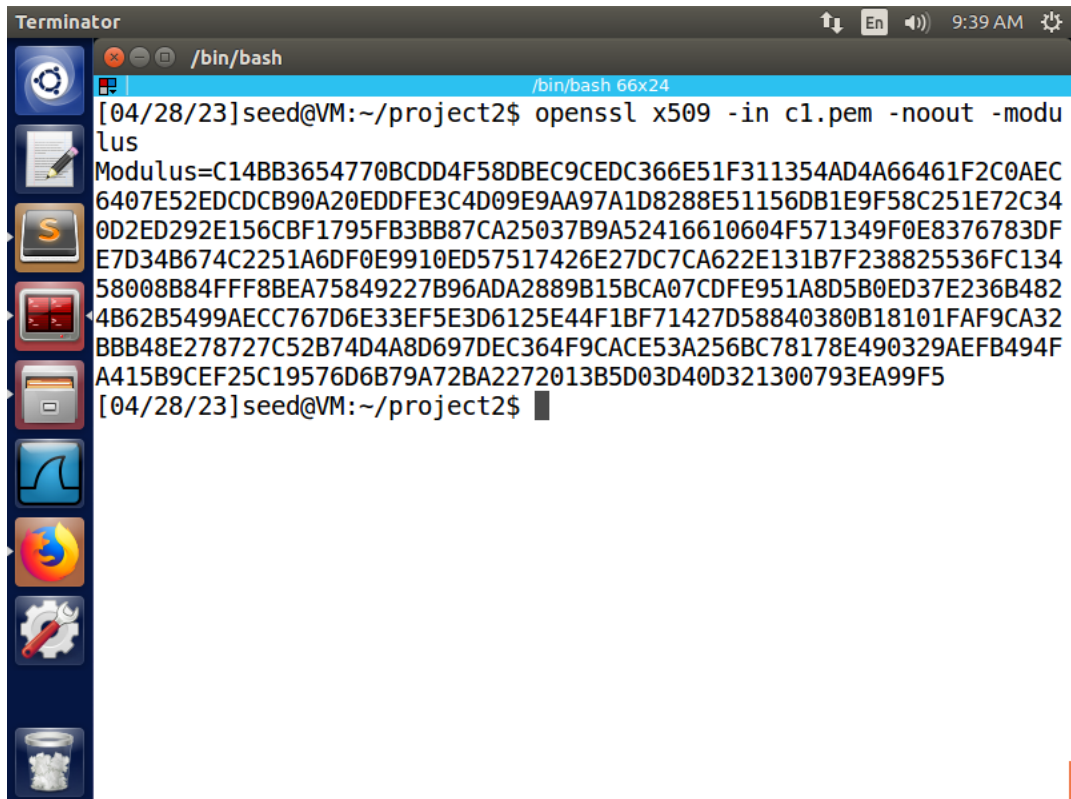
Εικόνα 6.2.1. Λήψη πιστοποιητικού του server

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



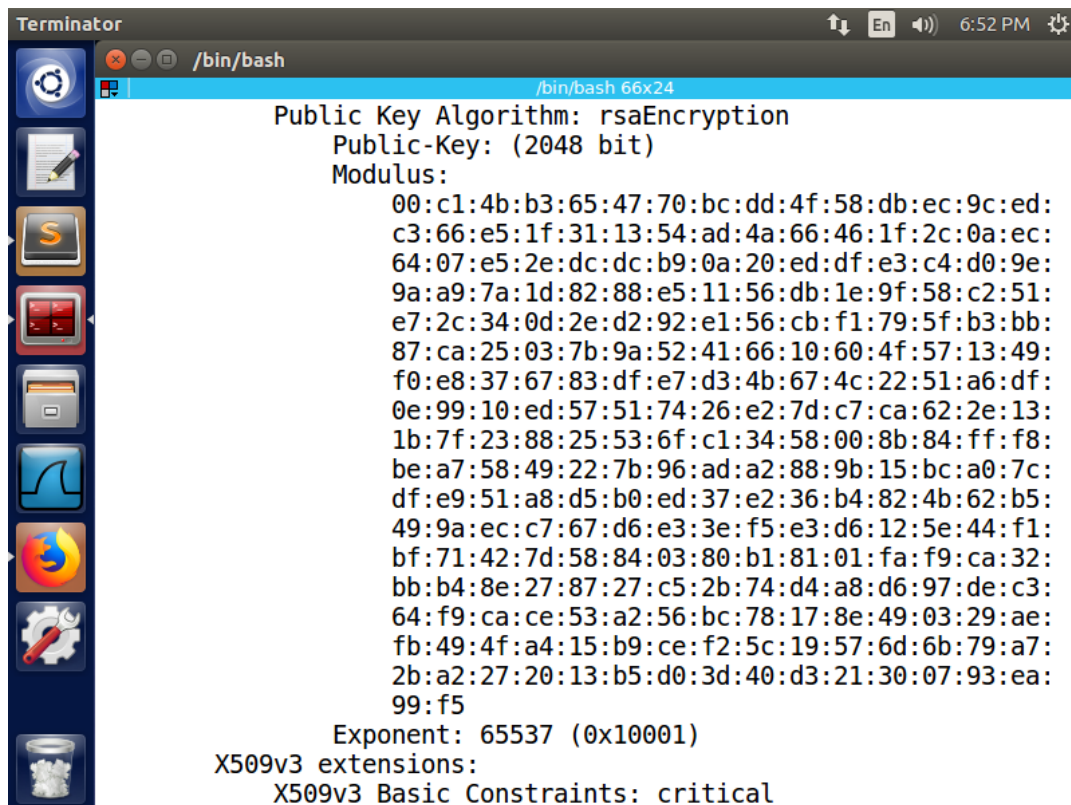
Εικόνα 6.2.2. Λήψη πιστοποιητικού του εκδότη

Βήμα 2



Εικόνα 6.2.3. Εξαγωγή του modulo N από το πιστοποιητικό του εκδότη

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

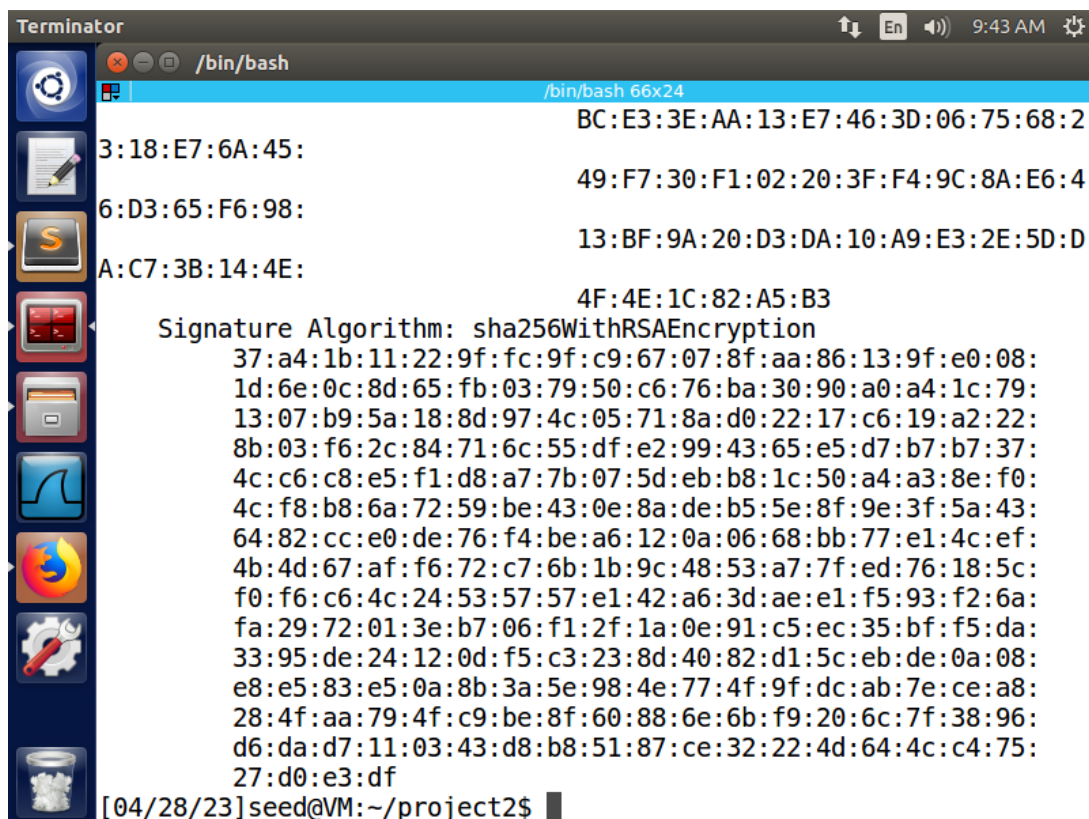


The screenshot shows a Terminator terminal window with a dark theme. The title bar includes the name 'Terminator' and system icons for volume, network, and time (6:52 PM). The terminal prompt is '/bin/bash'. The output displays the details of a public key algorithm: 'rsaEncryption' with a 'Public-Key: (2048 bit)'. The 'Modulus' is shown as a long hexadecimal string. The 'Exponent' is '65537 (0x10001)'. At the bottom, it shows 'X509v3 extensions:' and 'X509v3 Basic Constraints: critical'. On the left side of the terminal, there is a vertical toolbar with various icons for file operations and system utilities.

```
Terminator 6:52 PM
/bin/bash
Public Key Algorithm: rsaEncryption
Public-Key: (2048 bit)
Modulus:
00:c1:4b:b3:65:47:70:bc:dd:4f:58:db:ec:9c:ed:
c3:66:e5:1f:31:13:54:ad:4a:66:46:1f:2c:0a:ec:
64:07:e5:2e:dc:dc:b9:0a:20:ed:df:e3:c4:d0:9e:
9a:a9:7a:1d:82:88:e5:11:56:db:1e:9f:58:c2:51:
e7:2c:34:0d:2e:d2:92:e1:56:cb:f1:79:5f:b3:bb:
87:ca:25:03:7b:9a:52:41:66:10:60:4f:57:13:49:
f0:e8:37:67:83:df:e7:d3:4b:67:4c:22:51:a6:df:
0e:99:10:ed:57:51:74:26:e2:7d:c7:ca:62:2e:13:
1b:7f:23:88:25:53:6f:c1:34:58:00:8b:84:ff:f8:
be:a7:58:49:22:7b:96:ad:a2:88:9b:15:bc:a0:7c:
df:e9:51:a8:d5:b0:ed:37:e2:36:b4:82:4b:62:b5:
49:9a:ec:c7:67:d6:e3:3e:f5:e3:d6:12:5e:44:f1:
bf:71:42:7d:58:84:03:80:b1:81:01:fa:f9:ca:32:
bb:b4:8e:27:87:27:c5:2b:74:d4:a8:d6:97:de:c3:
64:f9:ca:ce:53:a2:56:bc:78:17:8e:49:03:29:ae:
fb:49:4f:a4:15:b9:ce:f2:5c:19:57:6d:6b:79:a7:
2b:a2:27:20:13:b5:d0:3d:40:d3:21:30:07:93:ea:
99:f5
Exponent: 65537 (0x10001)
X509v3 extensions:
X509v3 Basic Constraints: critical
```

Εικόνα 6.2.4. Εξαγωγή του δημοσίου κλειδιού ε από το πιστοποιητικό του εκδότη

Βήμα 3

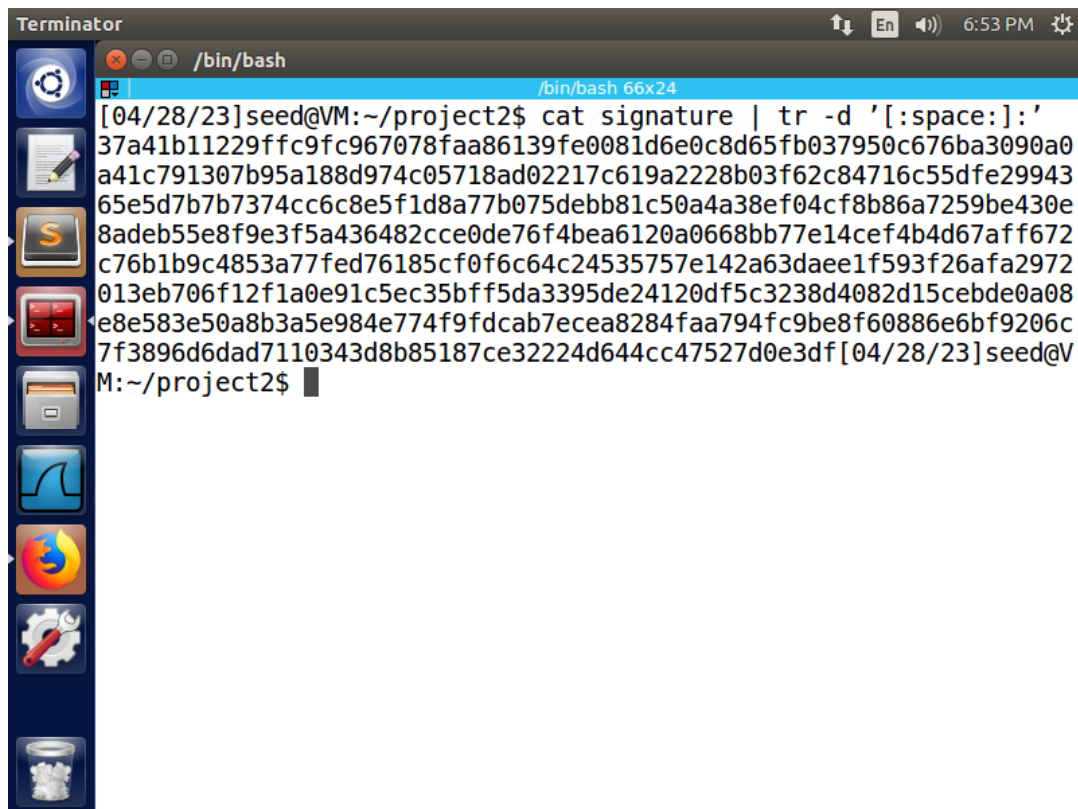


The screenshot shows a Terminator terminal window with a dark theme. The title bar includes the name 'Terminator' and system icons for volume, network, and time (9:43 AM). The terminal prompt is '/bin/bash'. The output displays the details of a signature algorithm: 'sha256WithRSAEncryption'. The 'Signature' is shown as a long hexadecimal string. At the bottom, it shows '[04/28/23]seed@VM:~/project2\$'. On the left side of the terminal, there is a vertical toolbar with various icons for file operations and system utilities.

```
Terminator 9:43 AM
/bin/bash
BC:E3:3E:AA:13:E7:46:3D:06:75:68:2
49:F7:30:F1:02:20:3F:F4:9C:8A:E6:4
13:BF:9A:20:D3:DA:10:A9:E3:2E:5D:D
4F:4E:1C:82:A5:B3
Signature Algorithm: sha256WithRSAEncryption
37:a4:1b:11:22:9f:fc:9f:c9:67:07:8f:aa:86:13:9f:e0:08:
1d:6e:0c:8d:65:fb:03:79:50:c6:76:ba:30:90:a0:a4:1c:79:
13:07:b9:5a:18:8d:97:4c:05:71:8a:d0:22:17:c6:19:a2:22:
8b:03:f6:2c:84:71:6c:55:df:e2:99:43:65:e5:d7:b7:b7:37:
4c:c6:c8:e5:f1:d8:a7:7b:07:5d:eb:b8:1c:50:a4:a3:8e:f0:
4c:f8:b8:6a:72:59:be:43:0e:8a:de:b5:5e:8f:9e:3f:5a:43:
64:82:cc:e0:de:76:f4:be:a6:12:0a:06:68:bb:77:e1:4c:ef:
4b:4d:67:af:f6:72:c7:6b:1b:9c:48:53:a7:7f:ed:76:18:5c:
f0:f6:c6:4c:24:53:57:57:e1:42:a6:3d:ae:e1:f5:93:f2:6a:
fa:29:72:01:3e:b7:06:f1:2f:1a:0e:91:c5:ec:35:bf:f5:da:
33:95:de:24:12:0d:f5:c3:23:8d:40:82:d1:5c:eb:de:0a:08:
e8:e5:83:e5:0a:8b:3a:5e:98:4e:77:4f:9f:dc:ab:7e:ce:a8:
28:4f:aa:79:4f:c9:be:8f:60:88:6e:6b:f9:20:6c:7f:38:96:
d6:da:d7:11:03:43:d8:b8:51:87:ce:32:22:4d:64:4c:c4:75:
27:d0:e3:df
[04/28/23]seed@VM:~/project2$
```

Εικόνα 6.2.5. Εξαγωγή της ψηφιακής υπογραφής από το πιστοποιητικό του server

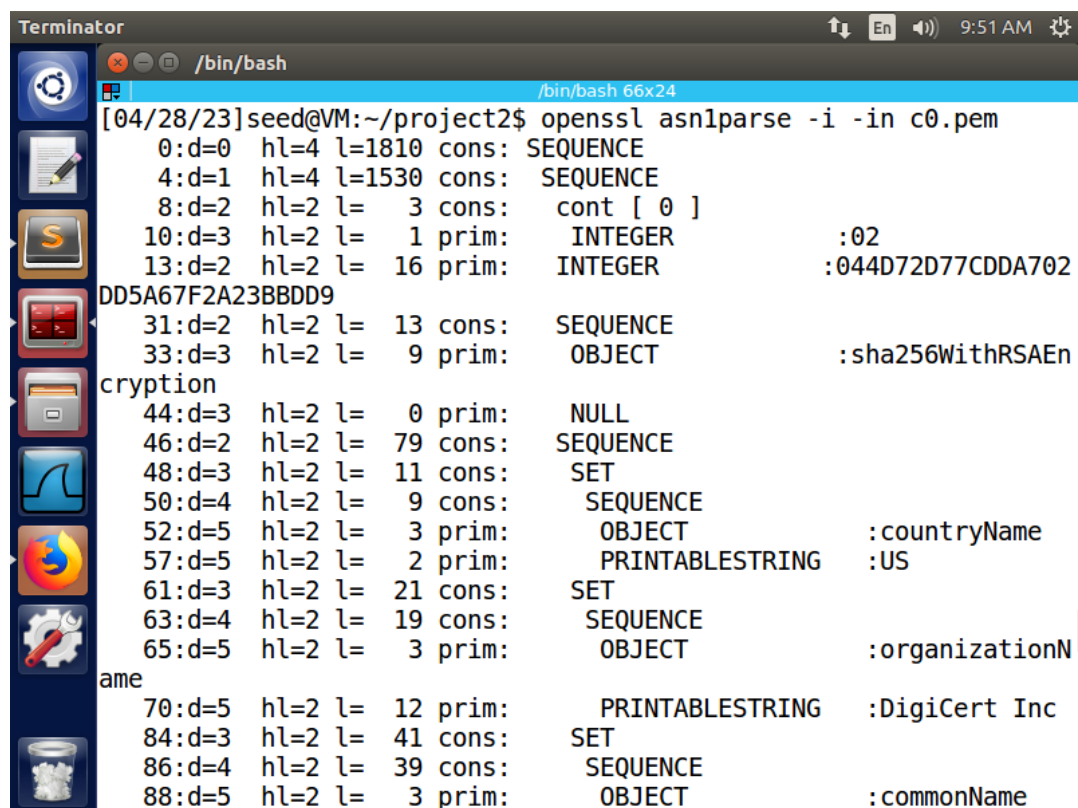
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ cat signature | tr -d '[:space:]:'
37a41b11229ffc9fc967078faa86139fe0081d6e0c8d65fb037950c676ba3090a0
a41c791307b95a188d974c05718ad02217c619a2228b03f62c84716c55dfe29943
65e5d7b7b7374cc6c8e5f1d8a77b075debb81c50a4a38ef04cf8b86a7259be430e
8adeb55e8f9e3f5a436482cce0de76f4bea6120a0668bb77e14cef4b4d67aff672
c76b1b9c4853a77fed76185cf0f6c64c24535757e142a63dae1f593f26afa2972
013eb706f12f1a0e91c5ec35bff5da3395de24120df5c3238d4082d15cebde0a08
e8e583e50a8b3a5e984e774f9fdcab7ecea8284faa794fc9be8f60886e6bf9206c
7f3896d6dad7110343d8b85187ce32224d644cc47527d0e3df[04/28/23]seed@V
M:~/project2$
```

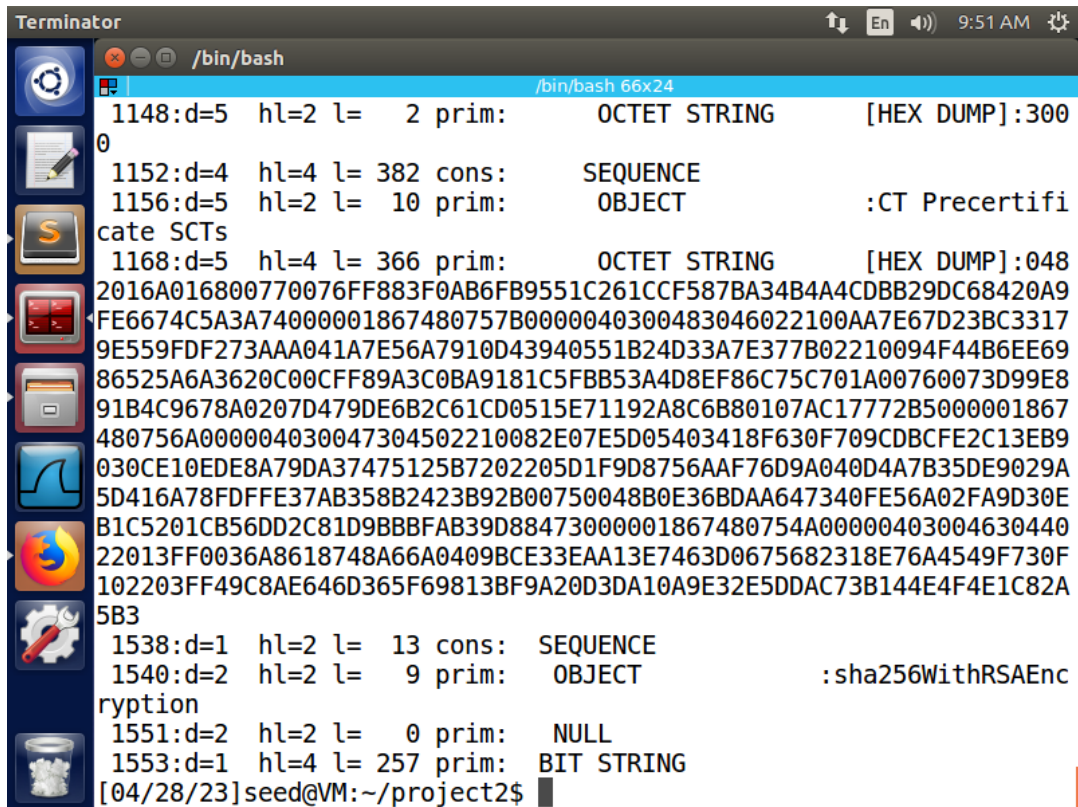
Εικόνα 6.2.6. Η ψηφιακή υπογραφή του πιστοποιητικού σε ενιαία σειρά

Βήμα 4



```
Terminator
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl asn1parse -i -in c0.pem
0:d=0  hl=4  l=1810 cons: SEQUENCE
4:d=1  hl=4  l=1530 cons: SEQUENCE
8:d=2  hl=2  l=  3 cons: cont [ 0 ]
10:d=3  hl=2  l=  1 prim: INTEGER           :02
13:d=2  hl=2  l= 16 prim: INTEGER           :044D72D77CDDA702
DD5A67F2A23BBDD9
31:d=2  hl=2  l= 13 cons: SEQUENCE
33:d=3  hl=2  l=  9 prim: OBJECT             :sha256WithRSAEn
ryption
44:d=3  hl=2  l=  0 prim: NULL
46:d=2  hl=2  l= 79 cons: SEQUENCE
48:d=3  hl=2  l= 11 cons: SET
50:d=4  hl=2  l=  9 cons: SEQUENCE
52:d=5  hl=2  l=  3 prim: OBJECT             :countryName
57:d=5  hl=2  l=  2 prim: PRINTABLESTRING    :US
61:d=3  hl=2  l= 21 cons: SET
63:d=4  hl=2  l= 19 cons: SEQUENCE
65:d=5  hl=2  l=  3 prim: OBJECT             :organizationN
ame
70:d=5  hl=2  l= 12 prim: PRINTABLESTRING    :DigiCert Inc
84:d=3  hl=2  l= 41 cons: SET
86:d=4  hl=2  l= 39 cons: SEQUENCE
88:d=5  hl=2  l=  3 prim: OBJECT             :commonName
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator /bin/bash
/bin/bash 66x24
1148:d=5 hl=2 l= 2 prim: OCTET STRING [HEX DUMP]:300
0
1152:d=4 hl=4 l= 382 cons: SEQUENCE
1156:d=5 hl=2 l= 10 prim: OBJECT :CT Precertifi
cate SCTs
1168:d=5 hl=4 l= 366 prim: OCTET STRING [HEX DUMP]:048
2016A016800770076FF883F0AB6FB9551C261CCF587BA34B4A4CDBB29DC68420A9
FE6674C5A3A74000001867480757B0000040300483046022100AA7E67D23BC3317
9E559FDF273AAA041A7E56A7910D43940551B24D33A7E377B02210094F44B6EE69
86525A6A3620C00CFF89A3C0BA9181C5FBB53A4D8EF86C75C701A00760073D99E8
91B4C9678A0207D479DE6B2C61CD0515E71192A8C6B80107AC17772B5000001867
480756A000004030047304502210082E07E5D05403418F630F709CDBCFC2C13EB9
030CE10EDE8A79DA37475125B7202205D1F9D8756AAF76D9A040D4A7B35DE9029A
5D416A78FDFFE37AB358B2423B92B00750048B0E36BDAA647340FE56A02FA9D30E
B1C5201CB56DD2C81D9BBBFAB39D88473000001867480754A00000403004630440
22013FF0036A8618748A66A0409BCE33EAA13E7463D0675682318E76A4549F730F
102203FF49C8AE646D365F69813BF9A20D3DA10A9E32E5DDAC73B144E4F4E1C82A
5B3
1538:d=1 hl=2 l= 13 cons: SEQUENCE
1540:d=2 hl=2 l= 9 prim: OBJECT :sha256WithRSAEnc
ryption
1551:d=2 hl=2 l= 0 prim: NULL
1553:d=1 hl=4 l= 257 prim: BIT STRING
[04/28/23]seed@VM:~/project2$
```

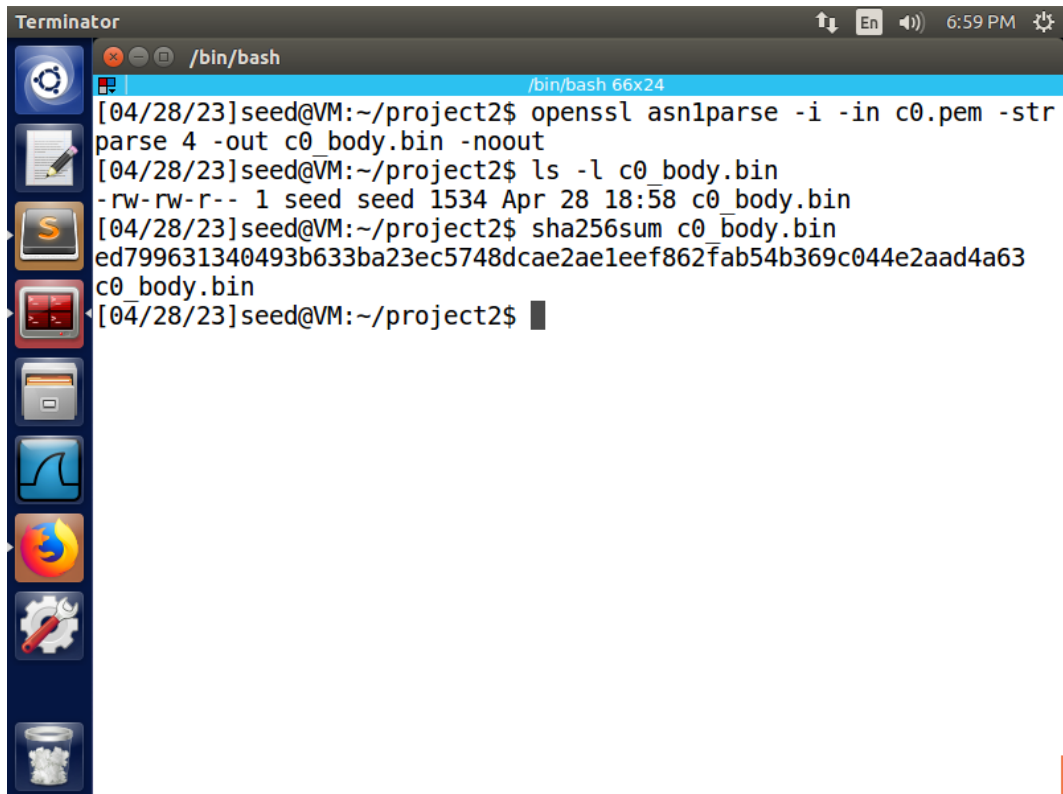
Εικόνα 6.2.7. Το σώμα του πιστοποιητικού



```
Terminator /bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl asn1parse -i -in c0.pem -str
parse 4 -out c0_body.bin -noout
[04/28/23]seed@VM:~/project2$ ls -l c0_body.bin
-rw-rw-r-- 1 seed seed 1534 Apr 28 09:54 c0_body.bin
[04/28/23]seed@VM:~/project2$
```

Εικόνα 6.2.8. Εξαγωγή του σώματος του πιστοποιητικού

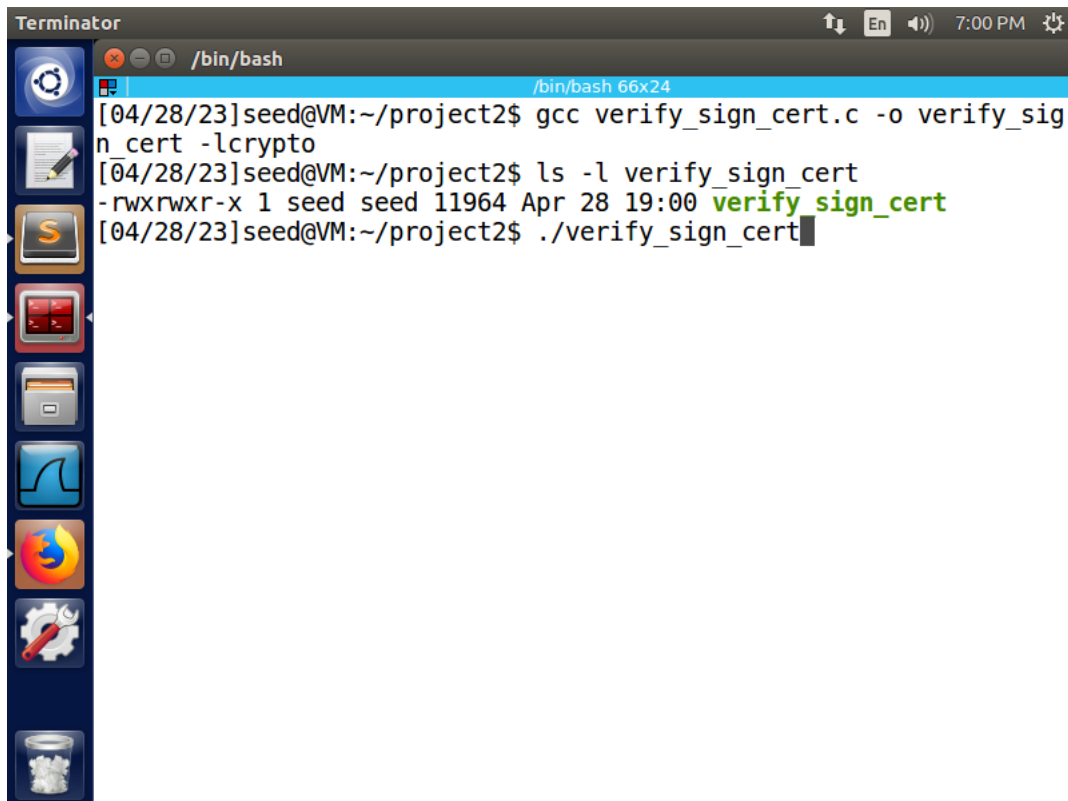
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator 6:59 PM
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ openssl asn1parse -i -in c0.pem -str
parse 4 -out c0_body.bin -noout
[04/28/23]seed@VM:~/project2$ ls -l c0_body.bin
-rw-rw-r-- 1 seed seed 1534 Apr 28 18:58 c0_body.bin
[04/28/23]seed@VM:~/project2$ sha256sum c0_body.bin
ed799631340493b633ba23ec5748dcae2ae1eef862fab54b369c044e2aad4a63
c0_body.bin
[04/28/23]seed@VM:~/project2$
```

Εικόνα 6.2.9. Υπολογισμός του hash

Βήμα 5



```
Terminator 7:00 PM
/bin/bash
/bin/bash 66x24
[04/28/23]seed@VM:~/project2$ gcc verify_sign_cert.c -o verify_sig
n_cert -lcrypto
[04/28/23]seed@VM:~/project2$ ls -l verify_sign_cert
-rwxrwxr-x 1 seed seed 11964 Apr 28 19:00 verify_sign_cert
[04/28/23]seed@VM:~/project2$ ./verify_sign_cert
```

Εικόνα 6.2.10. Μεταγλώττιση και εκτέλεση του προγράμματος verify_sign_cert.c

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



Σας ευχαριστώ για την προσοχή σας.

